

BiGym 2.0: Benchmarking Learned and Agent-Developed Policies for Humanoid Household Manipulation

Zexi Zhang*, Zecheng Zhu*, Zidong Chen, Zulkhuu Tuya and Stephen James

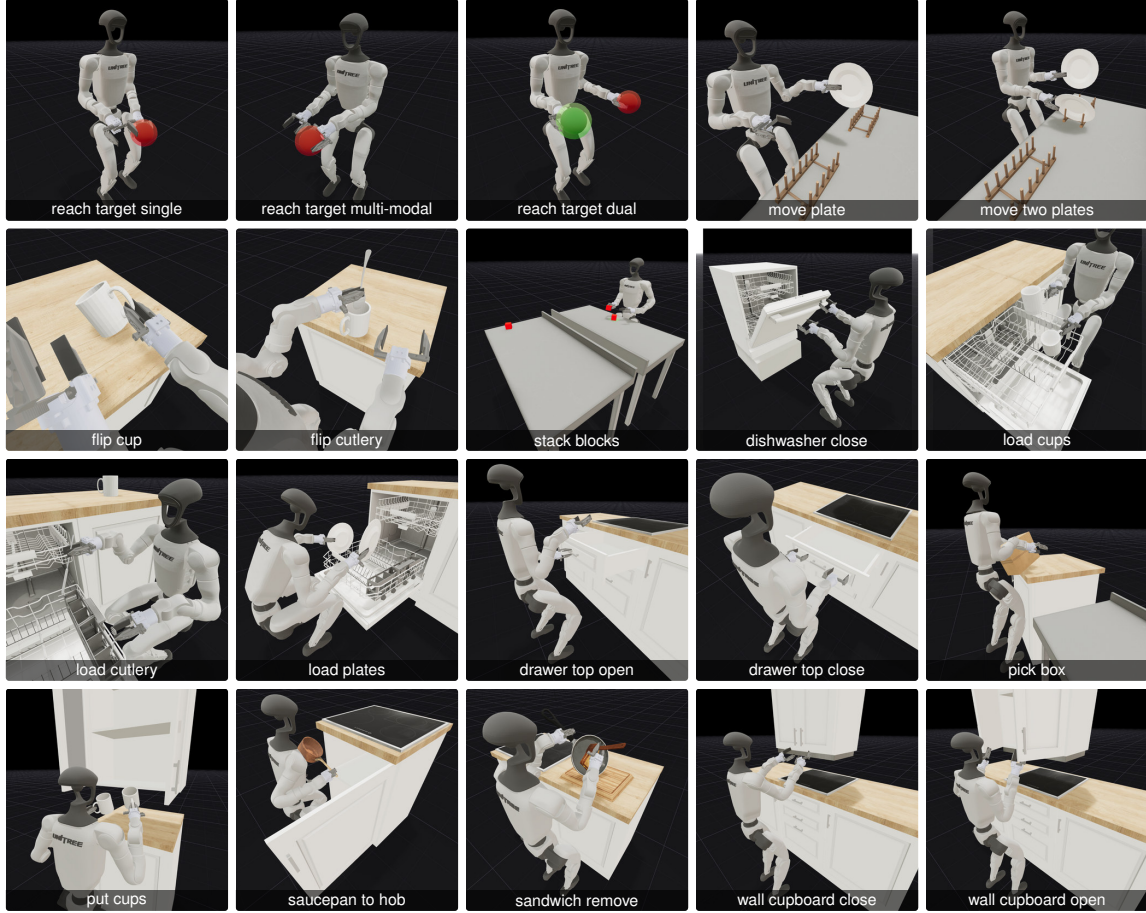


Fig. 1: BiGym 2.0: Auditing learned and agent-developed household manipulation on a walking humanoid. The suite evaluates data-driven policies and coding agents across 20 tasks spanning reaching, tabletop, dishwasher and kitchen-counter scenes. Tasks test hand choice, bimanual coordination, articulated fixtures, multi-object transport and cross-workspace block stacking. Human demonstrations, learned policies and agent programs execute through the same frozen whole-body controller, evaluating manipulation reliability under dynamic balance and footfall disturbances.

Abstract—Humanoid household manipulation requires the arms to act while the body balances, steps and changes posture. We present BiGym 2.0, an adaptation of BiGym for the Unitree G1 across 20 household tasks using a unified whole-body controller for demonstration and evaluation. The suite provides 60 native human virtual-reality demonstrations per task with synchronised multi-camera views and full-body execution records. We benchmark vision-language-action fine-tuning, imitation learning, demo-driven reinforcement learning, and cold-start coding agents given the interaction budget of online reinforcement learning. With the same onboard views, proprioception and whole-body controller for every method, vision-language-action fine-tuning has the highest nine-task mean, and agent-developed programs outperform every demo-driven reinforcement learning baseline on this mean and lead

on bimanual reaching. Cross-workspace stacking remains open, $\pi_{0.5}$ stays low on pick-box, and multi-object transport is hard for imitation learning, demo-driven reinforcement learning and coding agents. All environments, human demonstrations, and evaluation traces are open-sourced at <https://github.com/swirl-uk/BiGym2>.

I. INTRODUCTION

Humanoid robots are expected to perform household tasks that combine walking with manipulation. Carrying an object, reaching into a dishwasher and changing working height require the arms to act while the legs support and move the body. Today, robot capabilities are produced along two distinct routes: data-driven policy learning (VLAs, imitation learning and demo-driven RL) and agentic policy development (coding agents synthesising executable programs from

*Equal contribution. All authors are with the Department of Computing, Imperial College London, UK.

environment APIs). Recent simulation platforms support G1 loco-manipulation through learned controllers [2]–[4], but no current benchmark combines household tasks, native human demonstrations collected through active whole-body control, deterministic restoration of controller state, and common evaluation of learned and agent-developed policies. This combination matters because both routes must confront the same physical execution stack: unlike a wheeled base that executes velocity commands directly, a walking controller realises them through dynamic gaits that lag, oscillate and perturb the torso [5]–[7]. We ask what each route can reliably complete under this execution condition, where their capabilities diverge and where they fail together.

BiGym [8] provides the closest foundation: 40 H1 household tasks with human virtual reality (VR) demonstrations. It offers a 23-dimensional full-joint whole-body mode and a mobile-bimanual mode, but its released demonstrations use the latter. Its *floating base* directly commands body translation, height and yaw, with planar motion equivalent to an omnidirectional autonomous mobile robot (AMR) base (Fig. 2). This separation isolates mobile-bimanual learning from locomotion control. We instead study legged execution, where gait and balance dynamics couple directly into manipulation. We modify BiGym for the G1 by adapting task geometry, collecting demonstrations through active whole-body control, restoring controller state at evaluation, and supporting learned and agent-developed policies.

Coding agents synthesise robot policies by composing perception, kinematics and control APIs into executable programs [9]–[12]. We formulate a cold-start capability audit: given environment APIs and three synchronised camera views of a single human demonstration, can a coding agent develop reliable household programs within a bounded interaction budget? To match the learned policies, the agent takes three onboard 84×84 views and proprioception as input and outputs the same arm, gripper and body commands, with no camera calibration or inverse-kinematics tools. This cold-start setting measures how reliably an agent develops a program without

a pre-built skill library, and gives a baseline against which later work can measure skill accumulation.

We present BiGym 2.0, a G1 household loco-manipulation benchmark based on BiGym. Our contributions are: (i) adapting 20 tasks to the G1 workspace, integrating frozen GR00T-WBC walking and balance policies [1], and collecting 60 native human VR demonstrations per task through the same controller used for evaluation (Fig. 1, Section III); (ii) comparing VLA fine-tuning, imitation learning, online and offline demo-driven RL on nine representative tasks with deterministic evaluation that restores controller states (Table II, Section IV); (iii) auditing coding agents (GPT-6 Astra and Claude Opus 5.5) under matched observation and action interfaces (84×84 views, direct joint control, no IK tools), showing that their programs lead on bimanual reaching, where a program can control each arm separately, and evaluating a tool-augmented cold-start reference to quantify the benefit of classical robotics tooling (Section V); and (iv) showing that vision-language-action fine-tuning leads plate and cup transport but not box transport, that multi-object transport remains difficult for imitation learning, demo-driven reinforcement learning and code synthesis, and that cross-workspace stacking remains an open frontier (Sections IV and V).

II. RELATED WORK

Manipulation benchmarks. RLBench [13], LIBERO [14] and robomimic [15] support studies of task variation, learning from demonstrations and transfer, while RoboCasa [16], BEHAVIOR-1K [17] and ManiSkill3 [18] scale household scenes, activities and simulation throughput. MimicGen [19] and DexMimicGen [20] generate additional demonstrations from a small human set. BiGym [8] provides 40 household tasks with human demonstrations on the H1. BiGym 2.0 modifies its task layouts and control interface for legged execution on the G1, with demonstrations collected on the adapted system.

Whole-body control and teleoperation. ExBody [21], HOMIE [22], AMO [23], TWIST [24] and GR00T [1] provide interfaces for humanoid whole-body motion. HumanPlus [25],

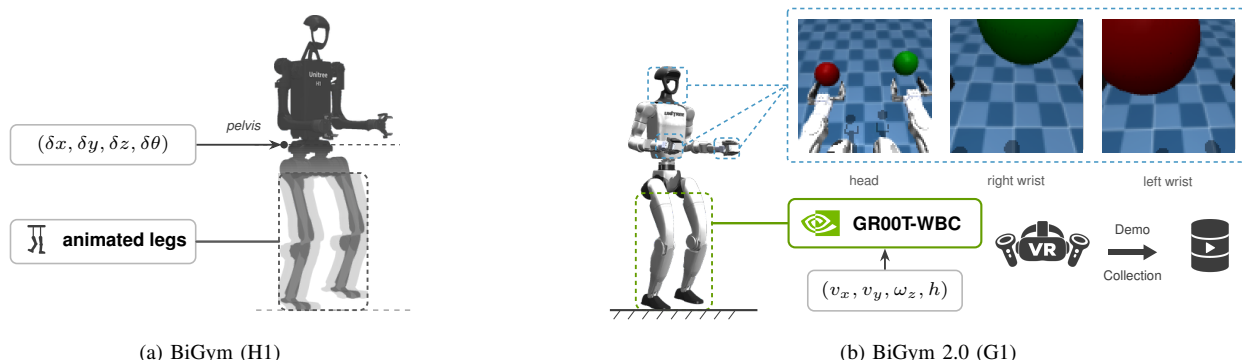


Fig. 2: From BiGym to BiGym 2.0. (a) BiGym offers full-joint control, but its released demonstrations use the mobile-bimanual mode shown: four increments set the H1 pelvis pose while the legs follow height-conditioned kinematic trajectories. (b) BiGym 2.0 interprets four commands as velocity and height references. Frozen GR00T-WBC policies [1] map them to leg and waist joint targets at 50Hz, with the feet supporting the robot; G1 also supports torso pitch. The same controller governs VR demonstration collection, evaluation and reset. Dashed boxes mark three onboard cameras and their recorded views from one dual-reaching episode at the policy input resolution, 84×84 . The panels are not to relative scale. Arm and gripper commands are omitted.

TABLE I: Humanoid manipulation benchmarks and demonstration resources. Task counts and reported policy families follow the cited papers. The two demonstration columns distinguish the availability of demonstrations from that of human demonstrations. Low-level execution names the interface or controller, not whether it is updated with the task policy.

Benchmark	Robot	Tasks	Low-level execution	Demonstrations		Policy families
				Any	Human	
HumanoidBench [28]	H1 / Digit	27 ^a	Full joints	–	–	RL
BiGym [8]	H1	40	FB / full joints ^b	✓	✓	IL, RL
Ego Humanoid Manipulation [29]	H1	12	Arm IK + hand PD	✓	✓	IL, VLA
HumanoidMimicGen [3]	G1	9	Learned controller	✓	✓ ^c	IL, VLA
SIMPLE [2]	G1	60 ^d	AMO / SONIC WBC	✓	✓ ^c	IL, VLA, WAM
HumanoidArena [4]	G1	7	Learned trackers	✓	✓	IL, VLA
BiGym 2.0 (ours)	G1	20	GR00T-WBC	✓	✓	IL, RL, VLA, agent

^a12 locomotion and 15 manipulation tasks. ^bFB: floating base, used for BiGym’s mobile-bimanual experiments and demonstrations. ^cIncludes both human and generated demonstrations. ^dSix tasks in the main comparison; six further tasks evaluated with one policy.

OmniH2O [26] and Open-TeleVision [27] support human teleoperation. We use a frozen GR00T-WBC controller during both VR collection and policy evaluation.

Humanoid learning environments. HumanoidBench [28] studies locomotion and manipulation with RL, while EgoVLA [29] provides a humanoid manipulation benchmark with teleoperated data. HumanoidMimicGen [3] studies data generation for a nine-task G1 loco-manipulation benchmark. SIMPLE [2] combines collection and rendering with evaluations of imitation, VLA and world-action model (WAM) policies, and HumanoidArena [4] evaluates hierarchical policies under visual, semantic and execution changes. FetchMan [30] studies visual reach-and-pick learning from synthetic demonstrations followed by RL. Humanoid Everyday [31] collects real-robot humanoid manipulation data across everyday activities. Our focus is a suite of household contact and transport tasks with native human demonstrations, supporting comparisons of VLA fine-tuning, imitation learning, demo-driven RL and agent-developed programs. Table I summarises the related settings.

Policy learning and program development. Pretrained VLAs such as OpenVLA [32], $\pi_{0.5}$ [33] and GR00T N1 [34] can be fine-tuned on demonstrations, while ACT [35] and DP [36] learn action sequences by imitation learning. CQN-AS [37], which extends coarse-to-fine RL [38] to action sequences, and DrQ-v2+ [38], [39] combine demonstrations with online interaction. DEAS [40] learns from a fixed offline dataset.

Language models generating robot policy code have an established research basis [9]. CaP-X studies how interface abstraction, interaction and perception affect code-policy performance [10]. RHO uses reflection and search to optimise policy repositories during development, deploying frozen code in its CaP-Bench experiments [11]. ASPIRE uses fine-grained execution traces to diagnose failures, validate repairs and accumulate reusable skills [12]. We follow this code-policy route to benchmark programs developed by GPT-6 Astra and Claude Opus 5.5 within a charged interaction budget on G1 household loco-manipulation. We isolate cold-start policy synthesis per task to measure development reliability, while using the suite to define the progression toward cumulative

skill reuse. Section III-E specifies the agent protocol and Section V the tool interfaces.

III. BiGYM 2.0

A. Loco-manipulation challenges and tasks

We organise BiGym 2.0 around two physical challenges and one evaluation question: (i) *Gait-perturbed interaction*: arms act while legs step and balance, coupling manipulation to dynamic footfall shock and base lag; (ii) *Coordinated multi-object transport*: carrying items across tables and counters demands continuous spatial tracking across changing visual viewpoints; (iii) *Cross-paradigm reliability*: how reliably learned policies and agent-written programs complete tasks when they share the same observations, actions and controller.

The suite instantiates these challenges across 20 tasks in four scene families (Fig. 1):

Reaching (3 tasks). Single reaching specifies the left hand, multi-modal reaching permits either hand, and dual reaching requires both hands to reach targets simultaneously, separating hand selection from bimanual coordination.

Tabletop (5 tasks). Single- and two-plate transport move plates between drying racks. Cup and cutlery flipping require reorientation. Block stacking combines cross-workspace transport with successive precise placements between tables.

Dishwasher (4 tasks). Door closing requires pushing racks and closing the door. Loading tasks place cups in the upper tray, cutlery in the basket and plates in the lower rack, combining fixture handling with constrained placement.

Kitchen counter (8 tasks). Four drawer and cupboard tasks provide fixture manipulation. Box pickup transports a large parcel to the counter. Further tasks store cups in cabinets, move a saucepan to the hob and transfer a sandwich with a spatula.

We adapt BiGym’s contact geometry to the G1 workspace. Plate racks are lowered and moved inward, and the dishwasher door/tray scene uses a 0.25 m plinth. For box pickup, the side table and counter are 0.50 m and 0.71 m high, respectively. These adapted layouts define the benchmark tasks.

Success is defined by the achieved scene state. Plate placement requires release, contact with the destination rack and the specified pose tolerance. Closing the dishwasher

```

from bigym.loco import make_gym

env = make_gym("move_plate")
agent = Agent(env.observation_space, env.action_space)
agent.ingest(env.get_demos(60))

obs, _ = env.reset()
for _ in range(100_000):
    action = agent.act(obs)
    nxt, reward, term, trunc, _ = env.step(action)
    agent.observe(obs, action, reward, nxt, term, trunc)
    agent.update()
    obs = nxt
    if term or trunc:
        obs, _ = env.reset()
env.close()

```

Fig. 3: Training an agent with BiGym 2.0. Load demonstrations, interact with the environment and update the agent through the Gymnasium API. The whole-body controller executes body commands within each environment step. Agent represents the user’s learning algorithm.

requires both trays and the door to be closed. Stacking requires a released three-block contact chain with height and planar-alignment constraints. These conditions must persist for 1 s. Appendix A gives the full predicates and reset distributions.

B. Whole-body execution

The robot is a 29-DoF Unitree G1 with two Dex1 parallel grippers in MuJoCo [41]. A high-level policy sends arm joint targets, gripper commands and body-motion commands. Frozen GR00T-WBC balance and walking policies convert the body commands into lower-body joint targets. The controller maintains balance while executing locomotion, height changes and torso pitch during manipulation.

Body commands specify forward and lateral velocity, yaw velocity and height, with optional torso pitch. Together with fourteen arm targets and two gripper commands, they form a 20- or 21-dimensional action. The high-level interface runs at 50 Hz, with 1 ms physics integration in the G1 scene. Reset restores simulator and controller state, then performs 200 high-level warmup steps. A Gymnasium interface supports demo-driven agent training (Fig. 3). Action layouts, bounds and controller settings are specified in Appendix B.

C. Human demonstrations

An operator supplies hand targets through VR and Mink inverse kinematics (IK) [42], and high-level body commands through thumbsticks. GR00T-WBC executes the body commands and maintains balance throughout collection. Collection requires the success condition to hold for 3 s. The training view retains the first 1 s of this hold, matching evaluation.

Each episode stores synchronised RGB, robot proprioception, normalised high-level actions, rewards and terminal indicators. Simulator state and controller snapshots are retained for replay and re-rendering, separately from the visual policy’s inputs. A native MuJoCo viewer and a Viser-based web viewer [43] support episode selection and frame stepping.

The suite provides 60 successful demonstrations for each of the 20 tasks, 1,200 episodes in total. As a loco-manipulation benchmark, BiGym 2.0 spans near-stationary manipulation

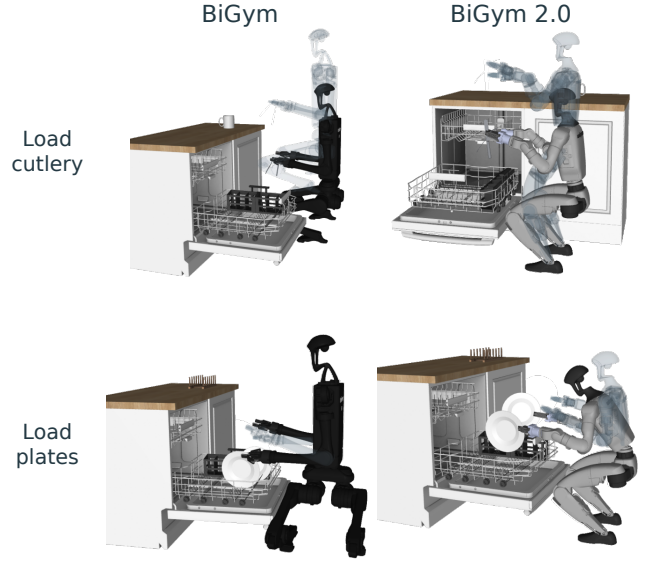


Fig. 4: Body posture during cutlery and plate loading. Left: BiGym; right: BiGym 2.0. Each panel overlays two states from one demonstration, with the earlier pose faint and the later pose opaque. Both systems adjust working height. G1 also pitches its torso forward during plate loading. Robots and task layouts follow their respective benchmarks.

and cross-workspace transport, with task-wise median planar base travel ranging from 0.06 to 8.10 m (5 Hz, 5 cm dead band). Tasks also involve changes in working height and torso posture. Fig. 4 illustrates height adjustment during cutlery loading and G1 torso pitch during plate loading. Appendix C gives the trajectory statistics and recording format.

D. Observations and evaluation

ACT, DP, CQN-AS, DrQ-v2+ and DEAS receive three 84×84 head/wrist RGB views, four-frame stacking and robot proprioception. $\pi_{0.5}$ receives the same 84×84 views and task instructions. Under the strict interface used in the main comparison, coding-agent programs receive the same three 84×84 views and the 50- or 56-dimensional proprioceptive vector, and send body, arm and gripper commands to the same controller in physical units (bounds in Appendix G). They receive no camera calibration, ray projection, inverse kinematics, privileged wrist positions or rate limiters. Section V separately evaluates a tool-augmented interface that adds these tools.

Each non-VLA learned-policy checkpoint is evaluated on 100 episodes with seeds $620\,000 + i$. Dishwasher closing and the two wall-cupboard tasks reset to a fixed initial state and do not consume these seeds. The other tasks randomise object placement per seed. Reset determinism is measured rather than assumed: without controller-state restoration, a seeded reset of a used environment differs from a fresh one by 9.5×10^{-2} in configuration and by 2.1×10^{-1} after 50 steps. With it enabled, the trajectories are bit-identical on the released substrate build. In the main comparison, the reported score averages the last five evaluated checkpoints

of each run, then reports the mean and standard error across 3 independent runs [44]–[47]. The nominal window is 80k–100k. Selecting each run’s best checkpoint would instead consult the evaluation seeds (Appendix E). The $\pi_{0.5}$ scores pool the last five checkpoints (26k–30k) with 50 episodes each, from one training run per task.

E. Policy families

For VLA fine-tuning, $\pi_{0.5}$ [33] uses supervised fine-tuning on the converted demonstrations and task instructions, with relative-delta actions and 16 executed actions per prediction. ACT [35] and Diffusion Policy (DP) [36] learn action sequences by imitation learning. Both train for 101k updates with batch size 256.

CQN-AS [37], DrQ-v2+ [38], [39] and DEAS [40] are demo-driven RL. The first two learn online with demonstrations, using 101k environment interactions and batches of 256 replay and 256 demonstration transitions. Successful online episodes can enter the demonstration buffer. DEAS learns offline over action sequences, using 101k updates on the fixed dataset. Our pixel-based implementation uses advantage-weighted regression (AWR) policy extraction. The online and offline budgets are counted in different units—interactions against updates—and are reported as such.

The reference execution recipe replans after each action for ACT, after eight actions for DP and after sixteen for DEAS. At 50 Hz these intervals are 0.02, 0.16 and 0.32 s. The non-VLA recipe uses random image shifts with padding 8 and masks non-base proprioception with probability 0.2 during training. Full method configurations are given in Appendix D.

GPT-6 Astra and Claude Opus 5.5 develop Python policies from a cold start (Fig. 5). For each task, a fresh session receives a task description, the environment API and three synchronised camera views of one demonstration, and carries nothing over from other tasks. A session may use 101,000 environment steps, the same count as the online learners, with 200 charged per reset. Development uses only the onboard views and the 60 demonstration seeds. The prompt allows NumPy, SciPy, OpenCV and Pillow, and forbids training networks, fitting regressions or building demonstration lookup tables. The submitted program runs without LLM calls on seeds $620\,000+i$, which the agent never sees, and ten episodes are repeated to check that outcomes are deterministic. Each agent has three sessions per task.

IV. CAPABILITY AUDIT

Table II audits learned policies and coding-agent programs across nine representative tasks under identical whole-body execution. Under the strict interface, $\pi_{0.5}$ has the highest nine-task mean, above imitation learning and both coding agents. Both coding agents still beat every demo-driven RL mean and rank alongside imitation learning. Opus leads multi-modal reaching. On dual reaching, both coding agents stay above every learned policy. $\pi_{0.5}$ leads single-plate transport, two-plate transport and cup loading. Pick-box transport remains higher for Opus and ACT than for $\pi_{0.5}$. Cross-workspace

stacking was not evaluated for $\pi_{0.5}$ and remains unsolved by ACT and CQN-AS.

A. Dual reaching and transport separate the methods

Every method solves drawer closing. Dual reaching separates the paradigms: every non-zero learned policy drops sharply from multi-modal reaching, where either hand suffices, to both hands at once. Both coding agents stay above every learned policy on dual reaching. On multi-modal reaching, Opus remains ahead of every learned policy, while Astra falls just below $\pi_{0.5}$. All six agent programs for dual reaching control the two arms separately, each with its own target and its own correction. A learned policy must instead reproduce the bimanual coordination from demonstrations.

Adding a second plate lowers success for ACT, DP, CQN-AS and DEAS, but not for $\pi_{0.5}$. Coding agents succeed on few plate-transport or cup-loading episodes. Opus succeeds on most pick-box episodes. No learned method leads on all nine tasks. Appendix F describes the contact fix for gait-induced grip slip and a DrQ-v2+ failure mode.

B. Checkpoint selection changes the ranking

On single-plate transport, among the three-run methods, ACT has the highest mean peak checkpoint while DP has the highest last-five mean, and $\pi_{0.5}$ ’s last-five score is higher than that DP mean. Selecting the best of 20 checkpoints, each estimated from 100 episodes, takes a maximum over noisy estimates. Table A7 lists peak and final scores per method.

C. The wider suite presents further challenges

Table III checks that the remaining eleven tasks are learnable from the provided demonstrations: ACT or CQN-AS reaches at least 10% on every task except cross-workspace stacking, which neither method solves in more than 1% of episodes. Section V examines the program policies and development budget for stacking.

V. CODING-AGENT POLICIES

Table II reports both agents under the strict interface and the protocol of Section III-E. This section examines how their programs behave and what classical robotics tools add.

A. Harnesses and interfaces

Each model runs in its official harness (Codex CLI v0.153.4, Claude Code v2.1.280) at high reasoning effort, in an isolated container with network access only to the model API (Appendix G).

The strict interface withholds tools, not the prior knowledge a model brings. Claude Opus 5.5 solved IK numerically in 24 of its 27 strict programs, and 22 of them use G1 link offsets recalled from Unitree’s public robot description rather than read from the simulator (Appendix G-C). No GPT-6 Astra program does either. The two agents reach similar nine-task means (Table II), so recalled kinematics did not remove the failures on contact-rich tasks.

The tool-augmented interface adds 640×480 views, camera calibration, pixel-to-ray projection, differential IK, rate limiters and world-frame wrist and base positions. We

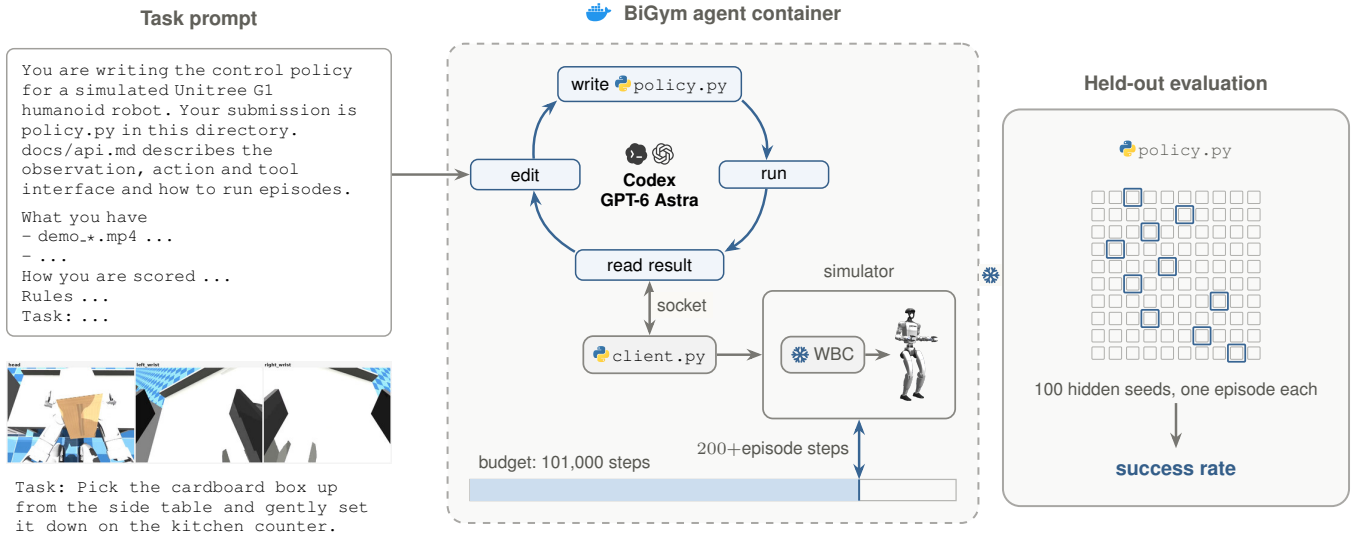


Fig. 5: Coding-agent development from a single demonstration. The agent receives three synchronised camera views of one human demonstration and onboard camera access, developing a program under the strict interface within a shared interaction budget. The frozen program is evaluated on 100 hidden seeds, with ten episodes repeated to check deterministic outcomes.

TABLE II: Success rates (%) on nine representative tasks. ACT, DP, CQN-AS, DrQ-v2+ and DEAS: mean $\pm$ standard error (SE) across three training runs, each averaging its last five checkpoints with 100 episodes each. $\pi_{0.5}$: one training run per task, pooled over its last five checkpoints with 50 episodes each. Coding agents: three independent development sessions each of Claude Opus 5.5 and GPT-6 Astra under the strict interface (84×84 onboard views, direct joint actions, no kinematics tools), 100 episodes per frozen program.

Task	VLA	Imitation learning		Demo-driven RL			Coding agent (strict)	
	$\pi_{0.5}$	ACT	DP	CQN-AS	DrQ-v2+	DEAS	Opus 5.5	Astra
Pick box	19	61 $\pm$ 4	10 $\pm$ 3	48 $\pm$ 2	0 $\pm$ 0	14 $\pm$ 3	71 $\pm$ 16	54 $\pm$ 22
Reach multi-modal	90	81 $\pm$ 1	84 $\pm$ 1	19 $\pm$ 16	0 $\pm$ 0	79 $\pm$ 2	100 $\pm$ 0	89 $\pm$ 7
Reach dual	58	30 $\pm$ 0	27 $\pm$ 0	3 $\pm$ 1	0 $\pm$ 0	34 $\pm$ 1	78 $\pm$ 10	89 $\pm$ 7
Drawer close	100	100 $\pm$ 0	100 $\pm$ 0	100 $\pm$ 0	99 $\pm$ 0	100 $\pm$ 0	100 $\pm$ 0	99 $\pm$ 1
Drawer open	99	92 $\pm$ 0	98 $\pm$ 0	89 $\pm$ 2	0 $\pm$ 0	79 $\pm$ 4	99 $\pm$ 1	99 $\pm$ 1
Move plate	65	60 $\pm$ 1	62 $\pm$ 4	48 $\pm$ 3	0 $\pm$ 0	27 $\pm$ 3	20 $\pm$ 2	14 $\pm$ 11
Move two plates	68	39 $\pm$ 0	18 $\pm$ 4	21 $\pm$ 3	0 $\pm$ 0	2 $\pm$ 1	15 $\pm$ 4	16 $\pm$ 11
Dishwasher close	79	93 $\pm$ 5	51 $\pm$ 2	78 $\pm$ 1	0 $\pm$ 0	27 $\pm$ 3	33 $\pm$ 33	100 $\pm$ 0
Dishwasher load cups	92	56 $\pm$ 3	51 $\pm$ 0	47 $\pm$ 7	0 $\pm$ 0	6 $\pm$ 2	28 $\pm$ 27	21 $\pm$ 4
Mean	75	68	56	50	11	41	61	65

TABLE III: ACT and CQN-AS success rates (%) on the remaining eleven tasks. Mean $\pm$ SE across three training runs, each averaging its last five checkpoints with 100 episodes each.

Task	ACT	CQN-AS
Reach single	89 $\pm$ 0	84 $\pm$ 1
Flip cup	62 $\pm$ 1	42 $\pm$ 9
Flip cutlery	46 $\pm$ 1	55 $\pm$ 2
Stack blocks	0 $\pm$ 0	1 $\pm$ 0
Load cutlery	11 $\pm$ 1	10 $\pm$ 3
Load plates	60 $\pm$ 1	34 $\pm$ 9
Put cups	38 $\pm$ 1	49 $\pm$ 2
Saucepan to hob	19 $\pm$ 1	32 $\pm$ 5
Remove sandwich	38 $\pm$ 2	17 $\pm$ 2
Wall cupboard close	100 $\pm$ 0	60 $\pm$ 31
Wall cupboard open	87 $\pm$ 7	7 $\pm$ 7

evaluate it for Astra only, in a separate set of sessions on a pre-release build that develop on seeds 0–199 (Table IV), to measure what classical robotics tools add to code synthesis.

Over its 27 strict sessions, Opus 5.5 read about four times as many tokens as Astra and cost more (\$420 versus \$296, API-equivalent). Per-session ledgers are released. In the pre-release sessions of Table IV, Astra’s strict sessions cost more than its tool-augmented ones, consistent with more trial and error without kinematics solvers.

B. Visual servoing under strict constraints, but contact vulnerability

Under the strict interface, both agents outperform the demo-driven RL methods on the nine-task mean. Astra’s programs show two execution modes: on reaching tasks, they close the loop in pixel space, thresholding the target in the wrist camera and moving the shoulder joints in proportion to its pixel offset, without 3D coordinates. For drawer closing, they replay an open-loop joint sequence that shoves the drawer shut without visual feedback, exploiting fixed contact geometry.

Both agents fail most plate-transport and cup-loading episodes, and results vary widely across sessions: a task is often solved in one session and failed in the others

TABLE IV: Coding-agent (GPT-6 Astra) performance and development cost. Strict vs. tool-augmented interfaces across nine tasks, from a separate set of sessions on a pre-release build of the environment, so strict scores differ from Table II. Task costs report mean per-session expenditure, with 27-session totals below.

Task	Success rate (%)			Cost/session (\$)	
	Strict	Tools	Δ	Strict	Tools
Reach multi	100 $\pm$ 0	100 $\pm$ 0	0	3.2	2.1
Reach dual	94 $\pm$ 2	99 $\pm$ 1	5	5.3	5.2
Drawer close	100 $\pm$ 0	100 $\pm$ 0	0	1.8	2.3
Drawer open	55 $\pm$ 27	99 $\pm$ 1	44	10.5	5.2
Move plate	39 $\pm$ 5	41 $\pm$ 26	2	10.8	11.7
Move 2 plates	0* $\pm$ 0	27 $\pm$ 25	27	25.3	15.4
Pick box	24 $\pm$ 20	34 $\pm$ 13	11	18.2	9.9
Dishwasher close	33 $\pm$ 33	100 $\pm$ 0	67	19.7	16.2
Load cups	32 $\pm$ 16	85 $\pm$ 8	53	14.0	10.4
Mean	53	76	23	12.1	8.7
Total (27 sessions)	—			326.3	235.1

*Reflects 1/300 successful episodes (0.33%, rounded to 0%).

(Appendix G-C), so a three-session mean reflects how often development finds a working strategy.

C. Impact of robotics tooling and cross-workspace stacking

Table IV compares the two interfaces. The full toolchain raises the nine-task mean, but unevenly: reaching and drawer closing barely change, so pixel-space servoing already suffices for open-space alignment, whereas drawer opening, dishwasher closing and cup loading gain the most.

Block stacking requires carrying blocks 0.98 m from one table to a pad on another and making successive precise placements. All 60 demonstrations contain two outbound carries and one return, with a median planar base path of 8.1 m. Each block must rest on the one below within a 5 cm planar tolerance. The released stack must hold for one second, and any block touching the floor terminates the episode. Under the tool-augmented interface, three Astra sessions reach 19%, 0% and 0%. The strongest program stores world-frame block and pad coordinates during initial perception and navigates closed-loop on the proprioceptive base pose, while grasp transitions remain time-indexed. With the 101,000-step budget and an 8,750-step episode limit, trials running to completion admit only eleven episodes, leaving narrow margins to repair long-horizon programs under bounded interaction.

VI. DISCUSSION AND LIMITATIONS

Both agent interfaces are cold-start: each task is developed in a fresh session that carries over no code, skills or notes from other tasks. We chose this to match the learned policies, each trained on a single task, so that neither route draws on other BiGym 2.0 tasks. Within a task, an agent sees one demonstration, whereas a learned policy trains on 60. We did not test whether agents improve by accumulating verified programs and reusing them across tasks. Recent work on fixed-base manipulation shows that such accumulation improves cross-task generalisation and reduces synthesis cost [12]. Our agent results are therefore a cold-start reference

rather than the ceiling of agent development, and the large variance across sessions (Appendix G-C) is one cost that accumulation may reduce. BiGym 2.0 provides the testbed to measure whether these gains carry over to humanoid locomotion under whole-body control.

All evaluations use simulation with one frozen whole-body controller per robot, characterising the joint policy-controller stack. Dishwasher closing and wall-cupboard tasks reset to fixed initial states as deterministic execution checks, while other tasks randomise object placements per seed. Because policies command high-level base velocities and pelvis height references rather than joint torques, execution inherently couples to the controller tracking accuracy and posture regulation under contact disturbances. Evaluating alternative whole-body controllers would help assess how policy performance depends on controller-specific dynamics. Agent results also rest on pretraining priors: Claude Opus 5.5 recalled the published kinematics of the G1, a well-documented robot (Section V), so agents may do worse on embodiments for which they hold no such prior. Future work includes diverse walking controllers, world-action models [48] and physical deployment.

VII. CONCLUSION

We present BiGym 2.0, a 20-task household locomotion benchmark adapting BiGym to the Unitree G1 with native human demonstrations under closed-loop whole-body control. We compare vision-language-action fine-tuning ($\pi_{0.5}$), imitation learning, demo-driven reinforcement learning, and cold-start coding agents (GPT-6 Astra and Claude Opus 5.5) under the same execution constraints. Vision-language-action fine-tuning attains the highest nine-task mean and leads plate and cup transport; agent-developed programs lead on bimanual reaching, where a program can control each arm separately. Cross-workspace stacking remains open, $\pi_{0.5}$ stays low on pick-box, and multi-object transport remains difficult for imitation learning, demo-driven reinforcement learning and agent-developed policies. Evaluation restores controller state, so results are reproducible, and the cold-start agent results give a baseline for work on skill accumulation. We plan to maintain and extend the suite.

REFERENCES

- [1] NVIDIA, “GR00T Whole-Body Control,” <https://github.com/NVlabs/GR00T-WholeBodyControl>, accessed: 2026-09-10.
- [2] S. Wei *et al.*, “SIMPLE: Simulation-Based Policy Learning and Evaluation for Humanoid Loco-manipulation,” *arXiv preprint arXiv:2606.08278*, 2026.
- [3] K. Lin *et al.*, “HumanoidMimicGen: Data Generation for Loco-Manipulation via Whole-Body Planning,” *arXiv preprint arXiv:2605.27724*, 2026.
- [4] T. Wang *et al.*, “HumanoidArena: Benchmarking Egocentric Hierarchical Whole-body Learning,” *arXiv preprint arXiv:2606.17833*, 2026.
- [5] P. Gysin, T. R. Kaminski, and A. M. Gordon, “Coordination of fingertip forces in object transport during locomotion,” *Exp. Brain Res.*, vol. 149, no. 3, pp. 371–379, 2003.
- [6] S. Sato *et al.*, “Drop Prevention Control for Humanoid Robots Carrying Stacked Boxes,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 4118–4125.
- [7] A. Huang, Z. Wu, S. Atar, Y. Zhi, and M. Yip, “SteadyTray: Learning Object Balancing Tasks in Humanoid Tray Transport via Residual Reinforcement Learning,” *arXiv preprint arXiv:2603.10306*, 2026.

- [8] N. Chernyadev, N. Backshall, X. Ma, Y. Lu, Y. Seo, and S. James, “Bi-Gym: A Demo-Driven Mobile Bi-Manual Manipulation Benchmark,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 4201–4217.
- [9] J. Liang *et al.*, “Code as policies: Language model programs for embodied control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023.
- [10] L. Fu *et al.*, “CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation,” *arXiv preprint arXiv:2603.22435*, 2026.
- [11] K. Elmaaroufi, J. Svegliato, S. Kalade, G. Schelle, S. A. Seshia, and M. Zaharia, “RHO: Your Coding Agent is Secretly a Robotician,” *arXiv preprint arXiv:2606.16458*, 2026.
- [12] R. Lu *et al.*, “ASPIRE: Agentic /Skills Discovery for Robotics,” *arXiv preprint arXiv:2607.00272*, 2026.
- [13] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison, “RLBench: The Robot Learning Benchmark & Learning Environment,” *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 3019–3026, 2020.
- [14] B. Liu *et al.*, “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.
- [15] A. Mandlekar *et al.*, “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 164, 2021, pp. 1678–1690.
- [16] S. Nasiriany *et al.*, “RoboCasa: Large-Scale Simulation of Household Tasks for Generalist Robots,” in *Proc. Robot. Sci. Syst. (RSS)*, 2024.
- [17] C. Li *et al.*, “BEHAVIOR-1K: A Benchmark for Embodied AI with 1,000 Everyday Activities and Realistic Simulation,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 205, 2022, pp. 80–93.
- [18] S. Tao *et al.*, “Demonstrating GPU Parallelized Robot Simulation and Rendering for Generalizable Embodied AI with ManiSkill3,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [19] A. Mandlekar *et al.*, “MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 229, 2023, pp. 1820–1864.
- [20] Z. Jiang *et al.*, “DexMimicGen: Automated Data Generation for Bimanual Dexterous Manipulation via Imitation Learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, pp. 16 923–16 930.
- [21] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, and X. Wang, “Expressive Whole-Body Control for Humanoid Robots,” in *Proc. Robot. Sci. Syst. (RSS)*, 2024.
- [22] Q. Ben, F. Jia, J. Zeng, J. Dong, D. Lin, and J. Pang, “HOMIE: Humanoid Loco-Manipulation with Isomorphic Exoskeleton Cockpit,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [23] J. Li, X. Cheng, T. Huang, S. Yang, R.-Z. Qiu, and X. Wang, “AMO: Adaptive Motion Optimization for Hyper-Dexterous Humanoid Whole-Body Control,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [24] Y. Ze *et al.*, “TWIST: Teleoperated Whole-Body Imitation System,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 305, 2025, pp. 2143–2154.
- [25] Z. Fu, Q. Zhao, Q. Wu, G. Wetzstein, and C. Finn, “HumanPlus: Humanoid Shadowing and Imitation from Humans,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 2828–2844.
- [26] T. He *et al.*, “OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 1516–1540.
- [27] X. Cheng, J. Li, S. Yang, G. Yang, and X. Wang, “Open-TeleVision: Teleoperation with Immersive Active Visual Feedback,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 2729–2749.
- [28] C. Sferazza, D.-M. Huang, X. Lin, Y. Lee, and P. Abbeel, “Humanoid-Bench: Simulated Humanoid Benchmark for Whole-Body Locomotion and Manipulation,” in *Proc. Robot. Sci. Syst. (RSS)*, 2024.
- [29] R. Yang *et al.*, “EgoVLA: Learning Vision-Language-Action Models from Egocentric Human Videos,” *arXiv preprint arXiv:2507.12440*, 2025.
- [30] O. Rayyan *et al.*, “FetchMan: Learning Visual Humanoid Loco-Manipulation Policies from Simulated Experiences,” *arXiv preprint arXiv:2608.17027*, 2026.
- [31] Z. Zhao *et al.*, “Humanoid Everyday: A Comprehensive Robotic Dataset for Open-World Humanoid Manipulation,” *arXiv preprint arXiv:2510.08807*, 2025.
- [32] M. J. Kim *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 2679–2713.
- [33] Physical Intelligence *et al.*, “ $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [34] NVIDIA, “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [35] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proc. Robot. Sci. Syst. (RSS)*, 2023.
- [36] C. Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Proc. Robot. Sci. Syst. (RSS)*, 2023.
- [37] Y. Seo and P. Abbeel, “Coarse-to-fine Q-Network with Action Sequence for Data-Efficient Reinforcement Learning,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [38] Y. Seo, J. Urug, and S. James, “Continuous Control with Coarse-to-fine Reinforcement Learning,” in *Proc. Conf. Robot Learn. (CoRL)*, ser. PMLR, vol. 270, 2024, pp. 2866–2894.
- [39] D. Yarats, R. Fergus, A. Lazaric, and L. Pinto, “Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [40] C. Kim, H. Lee, Y. Seo, K. Lee, and Y. Zhu, “DEAS: DEtached value learning with Action Sequence for Scalable Offline RL,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [41] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
- [42] K. Zakka, “Mink: Python inverse kinematics based on MuJoCo,” Jun. 2026, version 1.2.0. [Online]. Available: <https://github.com/kevinzakka/mink>
- [43] B. Yi *et al.*, “Viser: Imperative, Web-based 3D Visualization in Python,” *arXiv preprint arXiv:2507.22885*, 2025.
- [44] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep Reinforcement Learning That Matters,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, no. 1, 2018.
- [45] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep Reinforcement Learning at the Edge of the Statistical Precipice,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021.
- [46] C. Colas, O. Sigaud, and P.-Y. Oudeyer, “How Many Random Seeds? Statistical Power Analysis in Deep Reinforcement Learning Experiments,” *arXiv preprint arXiv:1806.08295*, 2018.
- [47] D. Snyder *et al.*, “Is Your Imitation Learning Policy Better than Mine? Policy Comparison with Near-Optimal Stopping,” in *Proc. Robot. Sci. Syst. (RSS)*, 2025.
- [48] S. Ye *et al.*, “World Action Models are Zero-shot Policies,” *arXiv preprint arXiv:2602.15922*, 2026.

APPENDIX A

TASKS AND SUCCESS PREDICATES

Fig. A1 shows the 20 tasks in the four scene families of Section III, and Table A1 gives each task’s action dimension, episode budget and success predicate. Every predicate must hold for 1 s (50 consecutive control steps). In the plate, cup, cutlery and block tasks an object touching the floor ends the episode as a failure; a robot fall is recorded but does not end the episode. The main comparison (Table II) uses nine of them; Table III reports ACT and CQN-AS on the other eleven.

TABLE A1: Episode budgets and success predicates of the 20 tasks. Horizon is the episode budget in seconds (control steps at 50 Hz). Every predicate must hold for 1 s. Normalised joint travel is 0 at one end of a joint’s range and 1 at the other. Tasks marked [†] are outside the nine-task main comparison.

Task	Act. dim	Horizon	Success predicate (held for 1 s)
<i>Reaching (3 tasks)</i>			
reach.target.single [†]	20	18 s (900)	Left end-effector (pinch-centre) site within 5 cm of the target.
reach.target.multi_modal	20	14 s (700)	Either end-effector site within 5 cm of the target.
reach.target.dual	20	14 s (700)	Each end-effector site within 5 cm of its own target, simultaneously.
<i>Tabletop (5 tasks)</i>			
move_plate	20	34 s (1700)	Plate within 5 cm of a target-rack slot, normal within 20° of the slot axis, touching the rack and not the table, released.
move_two_plates	21	46 s (2300)	Both plates satisfy the single-plate predicate.
flip.cup [†]	21	37 s (1850)	Mug upright within 5°, on the counter, released.
flip.cutlery [†]	21	39 s (1950)	Spoon within 50° of pointing down, touching the mug; spoon and mug released.
stack.blocks [†]	21	175 s (8750)	Bottom block on the pad, contact chain, each block ≥ 3 cm above and within 5 cm planar offset of the one below, all released.
<i>Dishwasher (4 tasks)</i>			
dishwasher.close	21	93 s (4650)	Door and both trays within 0.05 of closed (normalised joint travel).
dishwasher.load.cups	21	40 s (2000)	Both mugs touching the upper tray, released.
dishwasher.load.cutlery [†]	21	53 s (2650)	Knife and fork in the cutlery basket, released.
dishwasher.load.plates [†]	21	63 s (3150)	Both plates touching the lower tray, within 20° of the slot axis, released.
<i>Kitchen counter (8 tasks)</i>			
drawer.top.close	20	17 s (850)	Top drawer within 0.1 of closed (normalised joint travel).
drawer.top.open	20	27 s (1350)	Top drawer within 0.1 of fully open (normalised joint travel).
wall.cupboard.close [†]	21	34 s (1700)	Both doors within 0.1 of closed (normalised joint travel).
wall.cupboard.open [†]	21	29 s (1450)	Both doors within 0.1 of fully open (normalised joint travel).
pick_box	21	88 s (4400)	3 kg box from the 0.50 m side table onto the 0.71 m counter: in contact, bottom face within 3 cm of the top, centre over the counter, released.
put.cups [†]	21	60 s (3000)	Both cups touching the bottom shelf of the wall cupboard, released.
saucepan.to.hob [†]	21	93 s (4650)	Saucepan touching the hob, released.
sandwich.remove [†]	21	63 s (3150)	Sandwich touching the board with either face up within 10°; grasping the sandwich directly fails.

Reset distributions and scene layout. Reset distributions are task-specific. Seventeen tasks randomise the scene per seed: object positions by 1–5 cm and yaw by $\pm 15^\circ$ to $\pm 180^\circ$ depending on the task, reaching targets within a box around a nominal point, and, in the two drawer tasks, the robot pose ($x \in [0.12, 0.14]$ m, $y \in [-0.05, 0.05]$ m, yaw $\pm 5^\circ$) and the initial drawer opening. Dishwasher closing and the two wall-cupboard tasks reset to a fixed state.

Furniture heights follow each scene. In box pickup the side table and the counter are 0.50 m and 0.71 m high; the default kitchen counter is about 0.85 m; the two stacking tables are 0.70 m high with centres 0.70 m apart. The dishwasher-closing scene raises the dishwasher on a 0.25 m plinth. The G1 is 1.32 m tall.

APPENDIX B

ROBOT, CONTROLLER AND POLICY INTERFACES

A. Embodiment Specifications and Whole-Body Execution

BiGym 2.0 executes all human demonstrations, learned policies and agent-written programs on a simulated Unitree G1 humanoid with 29 actuated degrees of freedom (12 in the legs, 3 at the waist and 14 in the two 7-DoF arms) and two Dex1 parallel grippers. Frozen GR00T-WBC policies [1] receive the body commands at 50 Hz and output position targets for the 12 leg joints and the 3 waist joints. MuJoCo position actuators with the controller’s PD gains track these targets at the 1 ms physics step, 20 substeps per control step.

Passive pelvis tilt. The simulated base is a chain of planar slide and yaw joints, as in BiGym, extended with two passive, unactuated hinges for pelvis roll and pitch; every recorded demonstration uses them. With roll and pitch locked, a lateral



Fig. A1: The 20 tasks of BiGym 2.0. MuJoCo renders of one demonstration per task: the state after reset (left) and the final frame (right). The coloured rule above each pair marks the scene family, as in Fig. 1. The final frame marks the success criterion: the 5 cm tolerance sphere for reaching, the object’s recorded displacement for placement tasks and the measured joint extent against its threshold for articulated fixtures. Table A1 gives the full predicates.

command of $v_y = 0.25$ m/s produced $+0.164$ m/s of lateral and $+0.059$ m/s of unintended forward velocity. With the passive hinges, the response is $+0.1725$ m/s lateral and $+0.014$ m/s forward, against $+0.180$ m/s lateral for the upstream free-floating reference.

B. Action Representation and Controller Contract

Table A2 gives the 20- and 21-dimensional action layouts. The first four dimensions are body commands: forward velocity, lateral velocity, base height and yaw rate. The policy range $[-1, 1]$ maps to the command ranges recorded in the demonstration metadata: $v_x \in [-0.35, 0.35]$ m/s, $v_y \in [-0.25, 0.25]$ m/s, $z_{\text{base}} \in [0.40, 1.00]$ m and $\omega_z \in [-0.50, 0.50]$ rad/s. Fourteen tasks add a 21st dimension, an absolute torso-pitch reference $\theta_{\text{pitch}} \in [-0.20, 0.80]$ rad; the three reaching tasks, both drawer tasks and plate transport use 20 dimensions because their demonstrations were collected before this channel was added (Table A1).

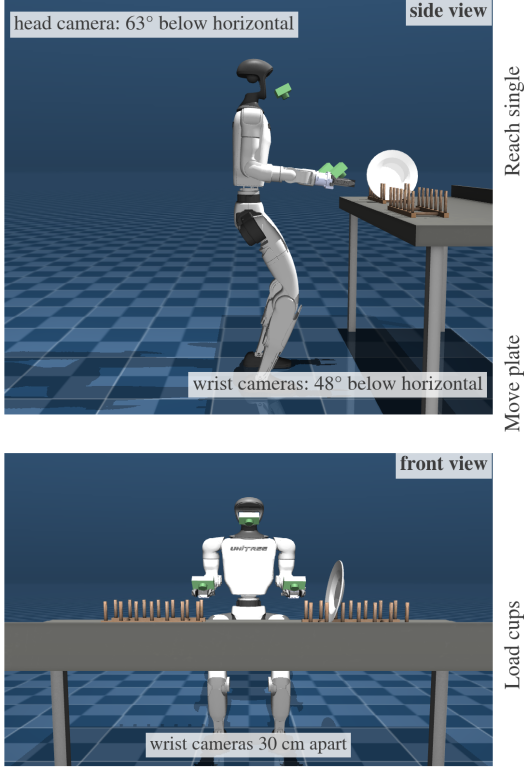
The remaining channels are absolute joint targets for the two 7-DoF arms, normalised over each joint’s range, and one closing command per Dex1 gripper (0 fully open, 1 fully closed). All baselines except $\pi_{0.5}$ predict these absolute targets. $\pi_{0.5}$ is trained on relative arm actions, expressed against the arm state at the start of each predicted chunk, with base and gripper channels kept absolute (Appendix D-C).

C. Proprioceptive State and Multi-Camera Observations

The non-VLA policies receive three synchronised 84×84 RGB views (Table A2, Fig. A2): a head camera on `torso_link` and one camera on each `wrist_yaw_link`, all with $\text{fovy} = 60^\circ$. At the plate-transport reference pose the head camera is pitched 63° below horizontal and the wrist cameras 48° , 30 cm apart. Four consecutive frames are stacked per camera. Proprioception is 50-dimensional, or 56-dimensional on torso-pitch tasks: positions and velocities of the arm, finger and

pelvis (x, y, z, yaw) joints, plus waist yaw, roll and pitch on 56-D tasks, followed by two gripper-closure scalars and the pelvis pose. It is stored raw and standardised with demonstration statistics when loaded.

(a) Camera placement



(b) Policy observations, 84×84 shown at $6 \times$

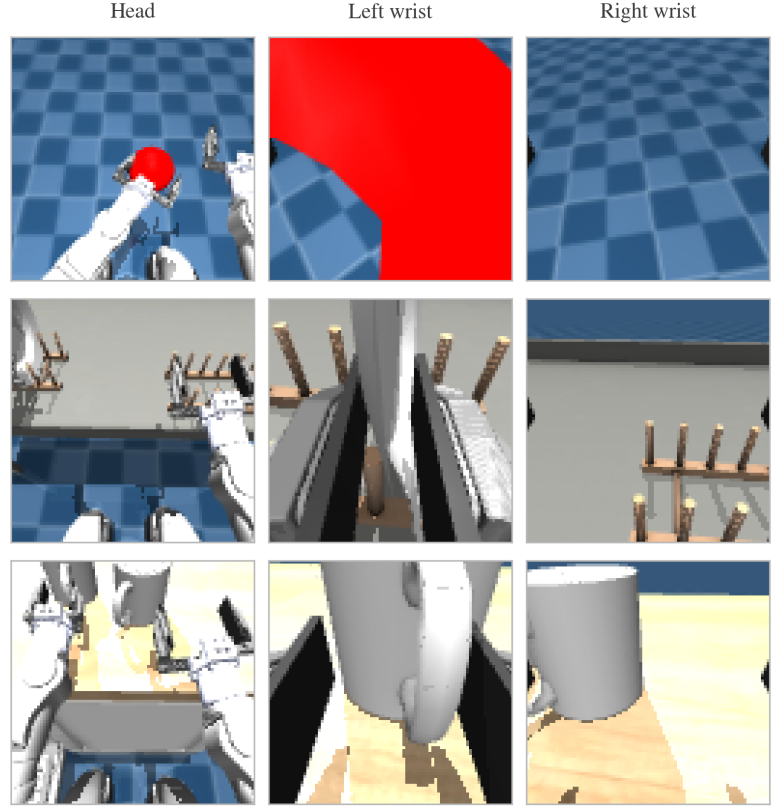


Fig. A2: Onboard cameras. (a) MuJoCo renders of the G1 at the start of a plate-transport demonstration; the green markers are the three cameras. The head camera sits on `torso.link` and each wrist camera on its `wrist_yaw.link`, 4.9 cm from the pinch centre between the finger pads. All three have a 60° field of view, horizontally and vertically, because the image is square. (b) Stored 84×84 observations from one demonstration of each of three tasks, taken when a finger pad first touches an object (for reaching, at success), upsampled $6 \times$ by nearest neighbour without other processing.

Reset determinism. The GR00T-WBC adapter keeps internal state across control steps, so a seeded reset alone does not reproduce a trajectory. With evaluation-time controller restoration disabled, a seeded reset of an environment that has already run a 60-step episode differs from the same reset of a fresh environment by 9.5×10^{-2} in configuration (maximum absolute `qpos` difference), growing to 2.1×10^{-1} within 50 steps. BiGym 2.0 therefore restores, at every evaluation reset, the controller state captured at the first reset. Fig. A4 shows the effect on open-loop replay of one plate-transport demonstration: 20 replays from the recorded engage snapshot (simulator and controller state) are identical at every step, whereas after a seed-only reset the final plate positions differ by up to 0.73 mm. All 40 replays succeed.

D. Simulation Throughput and Training Compute

We timed a single environment built by the benchmark protocol constructor in one process, with 1 ms physics and 20 substeps per 50 Hz control step, on an otherwise idle NVIDIA RTX 5090 of a workstation with an AMD Ryzen Threadripper PRO 7975WX (32 cores), using MuJoCo 3.8.1 with EGL rendering. The benchmark process was pinned to one CPU core while other training jobs ran on the remaining cores (load average 16 on 64 threads). Each configuration was stepped for 100 warm-up steps and then three repeats of 1,000 steps, either holding a fixed action or replaying demonstration actions; resets were timed separately. With GR00T-WBC in the loop and rendering disabled, the environment runs at 360–550 control steps/s, 7–11 times real time; with the benchmark observation of three 84×84 cameras it runs at 210–278 steps/s, 4–6 times real time (Table A3). Rendering the same cameras at 224×224 costs at most a further 8%. A reset, including the 200-step controller settle, takes 0.36–0.47 s.

TABLE A2: Action and observation layout of the G1. The standard configuration has 20 action and 50 proprioceptive dimensions; torso-pitch tasks (*) have 21 and 56. Command ranges are those recorded in the demonstration metadata.

Channel	Slice	Content	Physical range	Policy range
<i>Action (20-D; 21-D with torso pitch*), 50 Hz</i>				
Forward velocity v_x	[0]	Body command to GR00T-WBC	$[-0.35, 0.35]$ m/s	$[-1, 1]$
Lateral velocity v_y	[1]	Body command to GR00T-WBC	$[-0.25, 0.25]$ m/s	$[-1, 1]$
Base height z_{base}	[2]	Body command to GR00T-WBC	$[0.40, 1.00]$ m	$[-1, 1]$
Yaw rate ω_z	[3]	Body command to GR00T-WBC	$[-0.50, 0.50]$ rad/s	$[-1, 1]$
Torso pitch θ_{pitch}^*	[4]*	Waist pitch reference	$[-0.20, 0.80]$ rad	$[-1, 1]$
Left arm	[4:11] / [5:12]*	Absolute joint targets: shoulder P/R/Y, elbow, wrist R/P/Y	Joint range	$[-1, 1]$
Right arm	[11:18] / [12:19]*	Absolute joint targets, same order	Joint range	$[-1, 1]$
Grippers	[18:20] / [19:21]*	Left, right Dex1 command	0 open, 1 closed	$[-1, 1]$
<i>Proprioception (50-D; 56-D*), stored raw, standardised with demonstration statistics at load time</i>				
Joint positions q	[0:22] / [0:25]*	(Waist Y/R/P*,) left arm 7, left fingers 2, right arm 7, right fingers 2, pelvis x, y, z , yaw	rad, m	Standardised
Joint velocities $\dot{q}$	[22:44] / [25:50]*	Same joints as q	rad/s, m/s	Standardised
Gripper state	[44:46] / [50:52]*	Left, right closure	0 open, 1 closed	Standardised
Pelvis pose	[46:50] / [52:56]*	Pelvis x, y, z , yaw (simulator state)	m, rad	Standardised
<i>Images: three onboard RGB cameras, 84×84, four stacked frames</i>				
Head	View 1	On torso_link, fovy = 60°	uint8	Method-specific
Left wrist	View 2	On left wrist_yaw_link, fovy = 60°	uint8	Method-specific
Right wrist	View 3	On right wrist_yaw_link, fovy = 60°	uint8	Method-specific

Training to 101k updates or environment steps took, per run, a median of 3.3 h for CQN-AS, 5.9 h for ACT (12.4 h when two runs shared a GPU), 7.9 h for DP, 5.3 h for DEAS and 1.7 h for DrQ-v2+, each on one RTX 5090 of the same machine. These times exclude the independent checkpoint evaluation and depend on how the machine was shared.

TABLE A3: Single-environment throughput in control steps per second (mean $\pm$ std over three repeats of 1,000 steps; hold action / demonstration replay) and reset time (mean $\pm$ std, 30 resets). Measured on an idle RTX 5090, with the process pinned to one core of a Threadripper PRO 7975WX.

Task	Control steps/s			Reset (s)
	No render	3×84^2	3×224^2	
Move plate	399 / 360	223 / 210	211 / 196	0.47
Reach dual	528 / 550	278 / 268	256 / 270	0.36
Dishwasher close	457 / 405	232 / 210	225 / 204	0.43

APPENDIX C

HUMAN DEMONSTRATIONS AND DATASET FORMAT

A. VR Teleoperation

Operators wore a Meta Quest 3 headset connected through WiVRn. The headset shows a stereo MuJoCo render placed at the robot’s head camera, with the head shell hidden; this view is separate from the 84×84 policy camera. Mink quadratic-programming inverse kinematics [42] converts each tracked hand pose into seven arm-joint targets. The left thumbstick commands planar velocity; the right thumbstick commands yaw rate horizontally and base height vertically, the latter integrated over time. On torso-pitch tasks a left-stick click toggles a torso-pitch mode, and the triggers close the grippers. The environment steps at 50 Hz independently of the headset frame rate. The same frozen GR00T-WBC executes the body commands during collection and evaluation, so the demonstrations contain the controller’s tracking lag and gait-induced torso motion.

B. Demonstration Motion Statistics

Fig. A3 summarises the 1,200 demonstrations:

- **Duration:** the median episode lasts 14.4 s (range 2.9–87.4 s); per-task medians range from 3.4 s (reach_target_multi_modal) to 77.1 s (stack_blocks). The episode budgets in Table A1 are longer.
- **Base travel:** per-task median planar base travel ranges from 0.06 m (wall_cupboard_open) to 8.10 m (stack_blocks), where every demonstration makes two outbound carries and one return.

- **Height and posture:** pelvis height spans 0.38–0.78 m across episodes (commands 0.40–0.80 m); 11 tasks never change the height command. Only `dishwasher_load_plates` commands torso pitch (58 of 60 episodes, up to 0.8 rad), yet every episode of every task reaches a peak pelvis lean of 6.4°–27.9°.

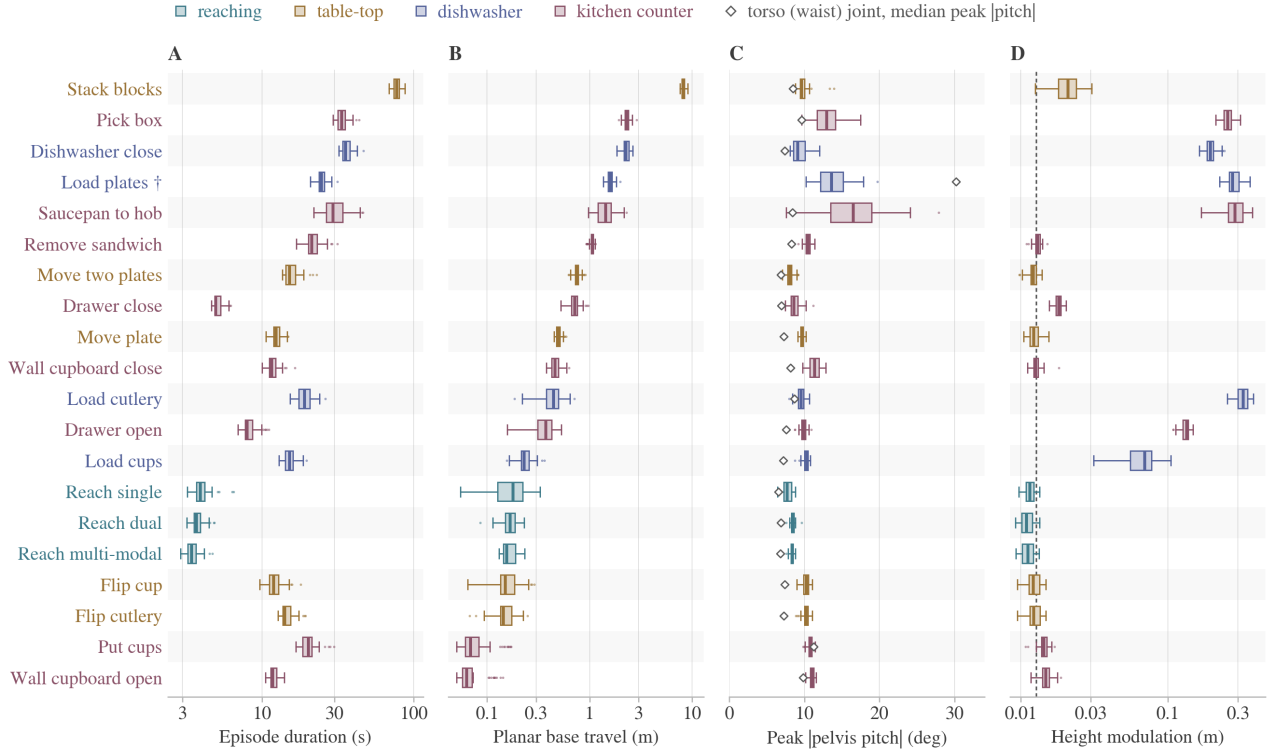


Fig. A3: Kinematic distributions of the 1,200 human VR demonstrations: 20 tasks $\times$ 60 successful episodes. **A:** episode duration (median 14.4 s). **B:** planar base travel, from 0.06 m near-stationary manipulation to 8.10 m cross-workspace transport. **C:** peak absolute pelvis lean within an episode, not a net or mean lean. **D:** physical pelvis height modulation, which is the executed travel rather than the command; the dashed line marks the 1.3 cm gait floor of the 11 tasks whose height command never moves. Boxes give the median and interquartile range, whiskers extend to $1.5 \times$ IQR and dots are episodes beyond them. Tasks are ordered by median base travel, so one row reads across all four panels; A, B and D are log-scaled. [†]Plate loading is the only task whose commanded torso-pitch action ever leaves 0 rad (58/60 episodes, peak 0.8 rad), yet every episode of all 20 tasks leans 6.4–27.9°.

C. Recording Format and Replay

The release contains 60 successful demonstrations for each of the 20 tasks, 1,200 episodes, each stored as a compressed NumPy `.npz` archive with JSON metadata. Per-step arrays include:

- `rgb_obs`: uint8 images of shape $(T, 3, 3, 84, 84)$, one per camera, in the order recorded in the metadata;
- `low_dim_obs`: raw 50- or 56-dimensional float32 proprioception;
- `action`: the normalised 20- or 21-dimensional policy action, with physical values in `raw_outer_action`;
- `reward` and `discount`;
- `full_qpos` and `full_qvel`: the complete simulator configuration and velocity;
- `lowerbody_command`, `height_command`, `torso_target` and `leg_joint_targets`: controller inputs and outputs.

Each episode also stores its seed and an engage snapshot taken when the operator takes control: `init_qpos`, `init_qvel`, `init_ctrl`, `init_qacc_warmstart` and the controller state (`lb_state.*`). Replaying the recorded actions from this snapshot reproduces the recorded trajectory; Fig. A4 shows 20 identical replays of one demonstration. Collection requires a 3 s success hold; the training view keeps the first 1 s of it, and the hold length is recorded in the metadata.

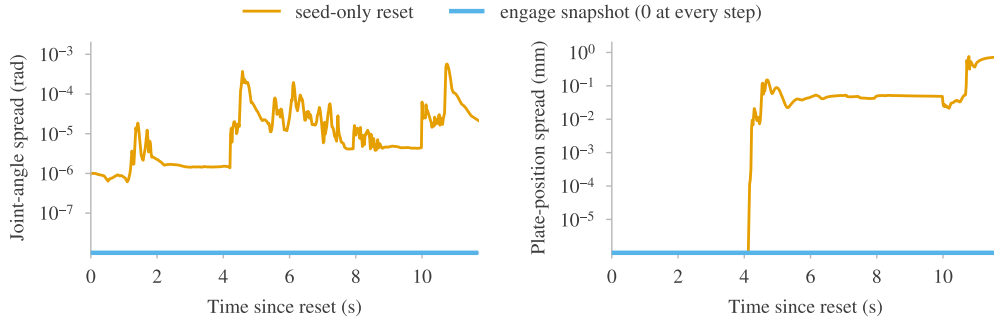


Fig. A4: Replay determinism. One `move_plate` demonstration (seed 18) replayed open-loop 20 times from its recorded engage snapshot and 20 times after a seed-only reset. Each curve is the largest difference to the first replay of the same condition, over the 29 actuated joint angles (left) and the plate position (right). From the snapshot, the replays are identical at every step. After a seed-only reset, the joint angles start 10^{-6} rad apart and the difference grows along the trajectory; the plate positions coincide until the gripper reaches the plate at about 4 s and end at most 0.73 mm apart. All 40 replays succeed.

APPENDIX D BASELINE IMPLEMENTATION AND TRAINING RECIPES

Table A4 lists the settings resolved from the run configurations of the main-table experiments; they were identical across the runs of each method.

A. Imitation Learning: ACT and Diffusion Policy

Action Chunking with Transformers (ACT). We use our JAX port of ACT [35]. Each camera view is encoded by an ImageNet-pretrained ResNet-18 with frozen batch normalisation; image tokens and a linear proprioception token enter a transformer encoder-decoder (4 encoder layers, 1 decoder layer, 8 heads, width 512) with a conditional VAE (32-dimensional latent, $\beta_{KL} = 10$). ACT predicts $K = 32$ actions, replans every step and fuses overlapping predictions by temporal ensembling with weights $\exp(-0.01 i)$. Training uses AdamW with learning rate 10^{-4} , 10^{-5} for the image encoder, weight decay 10^{-4} and batch 256 for 101k updates.

Diffusion Policy (DP). Each view is encoded by an ImageNet-pretrained ResNet-18 with frozen batch normalisation and spatial softmax. The image and proprioception features condition a 1D temporal U-Net (channels 256, 512, 1024) through FiLM. DP uses 100 DDPM steps in training and 10 DDIM steps at inference, predicts $K = 16$ actions and executes 8 before replanning. Training uses AdamW with learning rate 10^{-4} , 10^{-5} for the image encoder, weight decay 10^{-6} , gradient clipping at 1.0, an exponential moving average of the weights (0.9999, used for evaluation) and batch 256 for 101k updates.

B. Demo-Driven Reinforcement Learning: CQN-AS, DrQ-v2+ and DEAS

CQN-AS. CQN-AS [37] is a critic-only method over action sequences. Each camera has its own 4-layer CNN, and a GRU encodes the action sequence. Actions are discretised coarse to fine (3 levels of 5 bins per dimension), and the critic is a C51 distribution over 51 atoms on $[-2, 2]$. The policy predicts $K = 32$ actions, replans every step and uses the same temporal ensembling as ACT. Each update samples 256 online and 256 demonstration transitions. The loss weights the distributional TD term by 0.1 and the demonstration terms, a first-order stochastic dominance term and a margin loss (margin 0.1), by 0.9. Training uses AdamW with learning rate 5×10^{-5} and weight decay 0.1 over 101k environment steps, one update per step. Collection actions receive Gaussian noise of standard deviation 0.01 on all dimensions plus 0.03 on the first three body-command dimensions, and successful online episodes can enter the demonstration buffer.

DrQ-v2+. DrQ-v2+ [38], [39] is a single-step ($K = 1$) actor-critic with a deterministic actor and twin distributional critics (101-bin categorical on $[-2, 2]$, hidden width 1024). Each camera has its own 4-layer CNN, trained through the critic loss. It uses the same 256+256 sampling and collection noise as CQN-AS, an MSE behaviour-cloning term with weight 1.0, and AdamW with learning rate 10^{-4} and weight decay 0.1.

DEAS. DEAS [40] learns offline from the demonstrations. Twin critics use an HL-Gauss distribution (101 bins on $[0, 1]$); a distributional value function is fitted by expectile regression ($\tau = 0.7$); a deterministic actor predicts $K = 16$ actions, all executed, and is extracted by advantage-weighted regression ($\beta = 1$, weights clipped at 100). The discount is 0.9 within a chunk and 0.99 for bootstrapping. Each camera has its own 4-layer CNN. Training uses Adam with learning rate 10^{-4} , no weight decay and batch 256 for 101k updates.

C. Vision-Language-Action (VLA) Fine-Tuning: $\pi_{0.5}$

We fine-tune the released $\pi_{0.5}$ base checkpoint independently on each task with full supervised fine-tuning (Table A5). The model combines a PaliGemma vision-language backbone with a SigLIP-So400M image encoder and a flow-matching action expert. The task instruction (Table A6) and the proprioceptive state, discretised into 256 bins, enter as text tokens.

$\pi_{0.5}$ predicts a 50-step action chunk and executes the first 16 steps (0.32 s at 50 Hz) before replanning. Arm actions are relative to the arm state at the start of the chunk; base and gripper actions are absolute. Each task trains for 30k updates.

D. Shared Training Augmentation

All non-VLA baselines use the same training augmentation. Images are randomly shifted by up to 8 pixels, padding by edge replication, independently per sample and view. With probability 0.2 per sample, the entire non-base proprioceptive vector is set to zero across all stacked frames; after standardisation, zero is the demonstration mean. The pelvis entries are kept. Neither augmentation is applied at evaluation.

TABLE A4: Hyperparameters of the evaluated methods. Non-VLA values are resolved from the run configurations of the main-table experiments. The online methods (CQN-AS, DrQ-v2+) sample 256 replay and 256 demonstration transitions per update and perform one update per environment step.

Hyperparameter	ACT	DP	CQN-AS	DrQ-v2+	DEAS	$\pi_{0.5}$
Policy representation	Transformer CVAE	1D temporal U-Net (FiLM)	C2F C51 critic, GRU over chunk	Actor + distributional twin critic	Distributional critics + AWR actor	PaliGemma VLM + flow-matching expert
Visual encoder	ResNet-18 (ImageNet, frozen BN)	ResNet-18 (ImageNet, frozen BN), spatial softmax	4-layer CNN per camera	4-layer CNN per camera	4-layer CNN per camera	SigLIP-So400M
Proprioception input	Linear token	Projected, concatenated	Projected, concatenated	Projected, concatenated	Projected, concatenated	256-bin discretised text tokens
Action chunk (K)	32	16	32	1	16	50
Executed per prediction (H)	1	8	1	1	16	16
Temporal ensemble	Yes ($m = 0.01$)	No	Yes ($m = 0.01$)	No	No	No
Action representation	Absolute	Absolute	Absolute, discretised (3 levels $\times$ 5 bins)	Absolute	Absolute	Arm delta; base, gripper absolute
Optimiser	AdamW	AdamW	AdamW	AdamW	Adam	AdamW
Learning rate (policy)	10^{-4}	10^{-4}	5×10^{-5}	10^{-4}	10^{-4}	2.5×10^{-5}
Learning rate (visual encoder)	10^{-5}	10^{-5}	5×10^{-5} (same)	10^{-4} (same)	10^{-4} (same)	2.5×10^{-5} (same)
Learning-rate schedule	Constant	Constant	Constant	Constant	Constant	1k warmup, cosine to 2.5×10^{-6}
Weight decay	10^{-4}	10^{-6}	0.1	0.1	0	10^{-2}
EMA / target network	–	EMA 0.9999, grad. clip 1.0	Target $\tau = 0.02$	Hard copy every 100 updates	Target $\tau = 0.005$	–
Batch size	256	256	256 replay + 256 demo	256 replay + 256 demo	256	48
Frame stack	4	4	4	4	4	1
Training augmentation	–	Shift (pad 8, replicate); per-sample non-base proprioception mask, $p = 0.2$	Gaussian 0.01 + base	Gaussian 0.01 + base	–	None
Exploration noise	–	–	0.03	0.03	–	–
Loss	L1 + KL ($\beta = 10$)	ϵ -prediction MSE (100 DDPM train / 10 DDIM steps)	0.1 C51 TD + 0.9 (FOSD + margin 0.1)	Distributional TD + MSE BC ($\lambda = 1$)	HL-Gauss TD + expectile ($\tau = 0.7$) + AWR ($\beta = 1$)	Flow matching (MSE)
Training budget	101k updates	101k updates	101k env. steps	101k env. steps	101k updates	30k updates

TABLE A5: $\pi_{0.5}$ fine-tuning settings. One training run per task.

Setting	Value
Base checkpoint	Released $\pi_{0.5}$ (lerobot/pi05_base)
Adaptation	Full supervised fine-tuning (no LoRA)
Demonstrations	60 successful VR episodes per task
Cameras	Head, left wrist, right wrist; 84×84 RGB (model 224×224)
Proprioception	50-D (no torso pitch) or 56-D (pitch tasks)
Action	Relative arm delta; base and gripper absolute
Action dimension	20 (no pitch) or 21 (with torso pitch)
Chunk / executed	50 / 16 outer steps (0.32 s at 50 Hz)
Optimiser	AdamW, peak lr 2.5×10^{-5} , cosine to 2.5×10^{-6} , 1k warmup
Batch	48 (global)
Precision	bfloat16
Train seed	1000 (one run per task)
Training horizon	30k updates for every task

TABLE A6: Language instructions used during $\pi_{0.5}$ fine-tuning and evaluation. [†]Not in the nine-task main comparison.

Task	Instruction
Pick box	Pick up the box from the side table and place it on the counter.
Reach multi-modal	Reach the target with either wrist.
Reach dual	Reach the two targets, one with each wrist.
Drawer close	Close the top drawer of the kitchen cabinet.
Drawer open	Open the top drawer of the kitchen cabinet.
Move plate	Move the plate between two draining racks.
Move two plates	Move two plates simultaneously from one draining rack to the other.
Dishwasher close	Push back all trays and close the door of the dishwasher.
Dishwasher load cups	Move the cups from the table into the dishwasher’s upper tray.
Reach single [†]	Reach the target with the left wrist.

APPENDIX E

EVALUATION PROTOCOL AND CHECKPOINT SELECTION

Each evaluated checkpoint is rolled out for 100 episodes with seeds $620\,000 + i$, $i \in \{0, \dots, 99\}$, disjoint from the demonstration seeds (1–199). The score in Table II is the *last-five mean*: for each training run, the mean success over its last five saved checkpoints in the nominal 80k–100k window, 500 episodes per run. We report the mean and standard error across the three runs. $\pi_{0.5}$ has one training run per task; its score pools 50 episodes at each of its last five checkpoints (26k–30k) and has no across-run error bar.

Peak checkpoint versus last-five mean. Table A7 compares the last-five mean with the mean over runs of each run’s best checkpoint. With 100 episodes per checkpoint, the maximum over about 20 checkpoints is biased upwards, and choosing it consults the evaluation seeds. The gap Δ also contains genuine late-training decline, as on dishwasher closing, where DEAS runs peak at 97.0% on average and end at 27.2%. It is smallest for DP (+5.5 points on average) and largest for DEAS (+13.6). We report the last-five mean because it is fixed in advance.

TABLE A7: Peak checkpoint versus last-five mean on the nine main tasks. Peak: mean over three runs of each run’s best checkpoint (100 episodes per checkpoint). Last-5: the main-table score. $\Delta = \text{Peak} - \text{Last-5}$ in percentage points. DrQ-v2+ is omitted (99% last-five mean on drawer closing, 0% on the other eight tasks).

Task	ACT			DP			CQN-AS			DEAS		
	Peak	Last-5	Δ	Peak	Last-5	Δ	Peak	Last-5	Δ	Peak	Last-5	Δ
Pick box	71.7	61.3	+10.4	18.3	9.9	+8.4	55.0	47.6	+7.4	26.7	13.7	+13.0
Reach multi-modal	83.3	80.6	+2.7	87.0	83.7	+3.3	28.3	18.9	+9.5	85.7	79.3	+6.4
Reach dual	33.0	29.9	+3.1	30.7	26.8	+3.9	10.7	2.9	+7.8	40.0	33.8	+6.2
Drawer close	100.0	99.8	+0.2	100.0	100.0	+0.0	100.0	99.9	+0.1	100.0	100.0	+0.0
Drawer open	94.3	92.3	+2.1	99.3	98.1	+1.2	93.3	89.1	+4.2	90.7	78.7	+12.0
Move plate	70.0	60.1	+9.9	67.3	61.7	+5.7	53.3	48.2	+5.1	34.7	26.8	+7.9
Move two plates	46.0	38.7	+7.3	23.3	18.5	+4.9	30.7	21.1	+9.5	4.0	2.1	+1.9
Dishwasher close	100.0	93.3	+6.7	60.3	50.9	+9.4	100.0	78.1	+21.9	97.0	27.2	+69.8
Dishwasher load cups	68.0	55.6	+12.4	63.7	51.3	+12.4	59.3	46.9	+12.5	11.0	5.9	+5.1
Mean	74.0	67.9	+6.1	61.1	55.7	+5.5	59.0	50.3	+8.7	54.4	40.8	+13.6

APPENDIX F

CONTACT MODELLING AND A DRQ-v2+ FAILURE MODE

A. Gait-Induced Grip Slip

Walking produces periodic foot-contact impacts; in one replayed plate-transport demonstration, foot-force peaks recur at 3.45 Hz (median interval 0.29 s). Early plate-transport runs showed grasped plates slowly rotating in the gripper. We traced this to torsional friction at the Dex1 finger pads, which have sliding friction 1.5 and contact priority 1, so their parameters govern pad–plate contacts. In a reduced model with the gait disturbance amplified threefold, 6 s of transport accumulated 0.745 mm of slip with the original pad torsional friction of 0.01:

$$0.745 \text{ mm} \xrightarrow{\mu_{\text{torsion}}=0.05} 0.002 \text{ mm} \xrightarrow{\text{solref/solimp}} 0.000 \text{ mm} \quad (1)$$

Raising pad torsional friction to 0.05 reduced the slip to 0.002 mm. Giving the pads the plate’s constraint parameters ($\text{solref} = [0.004, 1]$, $\text{solimp} = [0.95, 0.99, 0.001]$), which contact priority had previously replaced with MuJoCo defaults, removed the remainder. No weld or attachment constraint is used.

B. Workspace Departure of DrQ-v2+

DrQ-v2+ scores 0% on eight of the nine main tasks. In 20 evaluation episodes of one DrQ-v2+ checkpoint on `reach_target_single` (seeds 620 000–620 000 + 19), the robot walked away from the workspace in every episode, backwards and sideways: the maximum pelvis displacement from the start ranged from 1.38 to 7.13 m (mean 4.20 m), and all 20 episodes timed out. These rollouts show how DrQ-v2+ fails; they do not isolate the lack of action chunking as the cause.

APPENDIX G CODING-AGENT DEVELOPMENT AND EXECUTION PROTOCOL

A. Cold-Start Development Protocol and Sandbox

We evaluate GPT-6 Astra (`gpt-6-astra`) in the official Codex CLI harness (v0.153.4) and Claude Opus 5.5 (`claude-opus-5-5`) in Claude Code (v2.1.280), both at high reasoning effort. The agent runs in a Docker container on an internal network whose only external route is a proxy allowlisted to the model provider’s API. It receives:

- 1) the task prompt (Listing A1);
- 2) the environment API documentation, including the observation and action layouts (Listing A2);
- 3) one human demonstration as three synchronised 84×84 MP4 files (`head`, `left_wrist`, `right_wrist`) with frame metadata.

Each session has a budget of 101,000 environment steps, with 200 steps charged per reset. Only NumPy, SciPy, OpenCV, Pillow and the Python standard library are installed. The prompt forbids learned components, meaning networks, regressions or lookup tables fitted to data; this rule is stated in the prompt and is not enforced by an automatic check.

B. Strict Interface versus Robotics Tooling

We run 27 development sessions per interface, three independent sessions for each of the nine tasks.

1. Strict interface (main comparison). The agent has the same observations and action channels as the learned policies: three 84×84 RGB views, 50- or 56-dimensional proprioception, and absolute arm joint targets with body commands, without rate limiting. It has no camera calibration, depth, pixel-to-ray projection or inverse kinematics, and develops on the 60 demonstration seeds. Body commands are clipped at ± 1.0 m/s and ± 1.0 rad/s rather than the demonstration bounds that limit the learned policies (Table A2). In the logged evaluation episodes, 95% of GPT-6 Astra’s control steps stay within those bounds.

2. Tool-augmented interface (reference). The agent additionally receives 640×480 images, pinhole camera calibration, pixel-to-ray projection, world-frame wrist and base positions, Mink damped differential inverse kinematics and a command rate limiter (6 rad/s with low-pass filtering), and develops on seeds 0–199.

Table A8 gives the per-session results, discussed in Section V. Listing A3 is the complete strict-interface program for multi-modal reaching, from these pre-release sessions.

Table A9 gives the per-session token use and API-equivalent cost of these pre-release strict-interface sessions; the most expensive single session cost \$31.23. The 27 tool-augmented sessions used 161.5 M input tokens and \$235.07.

TABLE A8: Per-session success and cost of GPT-6 Astra under the strict and tool-augmented interfaces, from the same sessions as Table IV. Each session’s frozen program is evaluated on 100 hidden seeds; Mean $\pm$ SE is across the three sessions. Cost is the mean per-session API-equivalent cost. On block stacking, ACT reaches 0.13% and tool-augmented Astra 6.3% (three-session mean; best session 19%).

Task	Strict Interface (84×84 RGB, direct joint targets)					Tool-Augmented Interface (640×480 , Calib, IK, DLS)					Δ
	Run 1	Run 2	Run 3	Mean $\pm$ SE	Cost (\$)	Run 1	Run 2	Run 3	Mean $\pm$ SE	Cost (\$)	
Reach multi-modal	100	99	100	100 ± 0	3.2	100	100	100	100 ± 0	2.1	0
Reach dual	98	91	94	94 ± 2	5.3	99	100	98	99 ± 1	5.2	+5
Drawer close	100	100	100	100 ± 0	1.8	100	100	100	100 ± 0	2.3	0
Drawer open	5	63	96	55 ± 27	10.5	100	98	99	99 ± 1	5.2	+44
Move plate	31	40	47	39 ± 5	10.8	93	18	12	41 ± 26	11.7	+2
Move two plates	0	0	1	$0^* \pm 0$	25.3	4	0	78	27 ± 25	15.4	+27
Pick box	63	8	0	24 ± 20	18.2	55	11	37	34 ± 13	9.9	+11
Dishwasher close	100	0	0	33 ± 33	19.7	100	100	100	100 ± 0	16.2	+67
Dishwasher load cups	0	44	51	32 ± 16	14.0	75	100	80	85 ± 8	10.4	+53
Mean	55.2	49.4	54.3	53	12.1	80.7	69.7	78.2	76	8.7	+23
Total (27 sessions)	1431/2700 (53.00%)				326.3	2057/2700 (76.19%)				235.1	—

*1 success in 300 episodes.

TABLE A9: Per-session token use and cost of GPT-6 Astra under the strict interface on the pre-release build, from the same sessions as Table IV. Three sessions per task.

Task	API-equivalent cost (\$)			Input tokens median (M)
	Run 1	Run 2	Run 3	
Reach multi-modal	2.72	4.33	2.65	1.5
Reach dual	3.09	6.72	6.08	3.7
Move plate	11.19	11.36	9.71	8.4
Move two plates	22.87	31.23	21.78	20.7
Dishwasher close	13.22	21.63	24.17	16.6
Dishwasher load cups	15.08	12.68	14.35	10.2
Drawer open	15.02	6.66	9.77	6.5
Drawer close	1.32	2.21	1.76	1.0
Pick box	10.85	28.68	15.20	11.8
Total (27 sessions)	326.33			237

C. Recalled Kinematics under the Strict Interface

The strict interface provides no inverse kinematics, and the simulator’s model files are not readable from the agent’s workspace. Claude Opus 5.5 nonetheless modelled the G1 arms itself. In 22 of its 27 scored programs, the arm chain uses link offsets that match the original release of Unitree’s public G1 description (`g1_29dof.urdf`) to five significant figures, for example the shoulder-pitch joint at (0.0039563, 0.10022, 0.23778) m from the torso. All seven arm offsets in its forward kinematics match that description; the simulator uses Unitree’s later revision with Dex1 grippers, which differs in two of them (shoulder pitch at $z = 0.24778$ m, wrist yaw at 0.051 m rather than 0.046 m), so the values were recalled from pretraining, not read from the simulator. In one session the agent’s closing message states that “the arm model uses G1 link lengths from memory of the robot description.” Of the 27 programs, 24 solve inverse kinematics numerically, 18 of them with `scipy.optimize.least_squares`; the three without are the drawer-closing sessions. No GPT-6 Astra program contains these values or solves inverse kinematics.

Listings A3 and A4 show the two routes on the same task, multi-modal reaching. Astra’s program thresholds the target colour in a wrist camera and moves the shoulder joints in proportion to its pixel offset. The Opus 5.5 program rebuilds the toolchain that the strict interface withholds: it models the head camera, lifts the target to 3D, walks to it and reaches it through forward kinematics from the recalled link offsets and a numerical IK. Both solve all 100 hidden seeds.

Table A10 lists every session. Opus 5.5 leads on multi-modal reaching and pick-box; Astra, without inverse kinematics, does better on dual reaching and dishwasher closing. Both agents fail most plate-transport and cup-loading episodes. The spread across sessions is large for both agents: a task is often solved in one session and failed in the others, so the three-session mean reflects how often a session finds a working strategy.

TABLE A10: Per-session success (%) of the coding agents under the strict interface. Each session’s frozen program is evaluated on 100 hidden seeds. *: the scored program solves inverse kinematics.

Task	Claude Opus 5.5			GPT-6 Astra		
	s1	s2	s3	s1	s2	s3
Pick box	44*	99*	71*	64	85	12
Reach multi-modal	100*	100*	100*	75	97	95
Reach dual	71*	99*	65*	100	93	75
Drawer close	100	100	100	96	100	100
Drawer open	99*	98*	100*	100	97	100
Move plate	21*	15*	23*	36	2	4
Move two plates	8*	15*	22*	0	38	9
Dishwasher close	0*	0*	100*	100	100	100
Dishwasher load cups	83*	1*	1*	18	16	30
Programs with IK	24 / 27			0 / 27		

D. Agent Prompt, Task Sentences, API Document and an Example Program

Listing A1 is the prompt of every strict-interface session, verbatim; only its last line, the task sentence, differs between tasks (Table A11). Tool-augmented sessions received the same template with 640×480 demonstration videos and training seeds 0–199. Listing A2 is the strict-interface API document, verbatim for the 20-dimensional tasks. On 21-dimensional tasks it adds the torso-pitch action at index 4 and the waist joints at the front of the proprioceptive vector; the tool-augmented

document additionally exposes inverse kinematics, camera calibration, pixel-to-ray projection and world-frame wrist and base positions. Entries 46–49 of the proprioceptive vector are the simulator’s pelvis joint positions (Table A2). Listing A3 is a complete submitted program.

Listing A1: Strict-interface prompt (verbatim text, line breaks reflowed; {task_sentence} from Table A11)

You are writing the control policy for a simulated Unitree G1 humanoid robot. Your submission is `policy.py` in this directory. `docs/api.md` describes the observation, action and tool interface and how to run episodes.

What you have

- `demo\_head.mp4`, `demo\_left\_wrist.mp4`, `demo\_right\_wrist.mp4`: one human demonstration of the task, recorded at the same time from the robot’s head, left wrist, right wrist cameras (84x84, 25 fps, frame k of each file is the same instant). `harness/demo.py` reads them as RGB arrays in the same layout as the live cameras and maps frames to control steps (see docs/api.md).
- Training seeds: the 60 seeds listed in `seeds.json`, the seeds the demonstration set was collected on. Object placements are drawn per seed; the hidden evaluation seeds draw from the same distribution.
- A budget of 101,000 environment steps for everything you run. Each reset costs 200 steps plus the steps of the episode.

How you are scored

- After you stop, `policy.py` is run once on 100 hidden seeds. Your score is the success rate over those 100 episodes; nothing you report is used. An episode in which the robot falls counts as failed.
- Episodes are re-run and must end the same way: no wall-clock time, no unseeded randomness.

Rules

- The policy is hand-written control logic. It may not contain learned components (no training a network, regression or lookup table from data). Take whatever you can from the demonstration video; tuning constants by running episodes is fine.
- Libraries: numpy, scipy, OpenCV (cv2), pillow and the Python standard library; nothing else is installed and nothing can be installed. No network, no changes to the harness. The simulator is reachable only through the client.
- The robot’s own cameras (`head`, `left\_wrist`, `right\_wrist`) and its body state are the only view of the scene, for you while developing as much as for the policy. There is no outside camera.
- Write code to files and run them with `./python file.py`; inline `python -c '...'` is refused by the sandbox and only wastes a turn.

Task: {task_sentence}

TABLE A11: Task sentences ending the agent prompt, identical under both interfaces. Agent sessions were run only on these tasks. The sentences were written for the agent and differ from the $\pi_{0.5}$ instructions (Table A6). [†]Tool-augmented sessions only.

Task	Task sentence
Pick box	Pick the cardboard box up from the side table and gently set it down on the kitchen counter.
Reach multi-modal	Touch the red sphere with either hand.
Reach dual	Touch the red sphere with your left hand and the green sphere with your right hand at the same time.
Drawer close	Close the open top drawer of the kitchen cabinet.
Drawer open	Open the top drawer of the kitchen cabinet.
Move plate	Take the plate out of the left dish rack and gently stand it in the right dish rack.
Move two plates	Take both plates out of the left dish rack and gently stand them in the right dish rack.
Dishwasher close	Push the dishwasher racks in and close the door.
Dishwasher load cups	Gently place the two mugs from the counter into the upper rack of the dishwasher.
Stack blocks [†]	Move the blocks to the other table and stack them on the green pad.

Listing A2: Strict-interface API document docs/api.md for 20-dimensional tasks (verbatim text, line breaks reflowed)

```
# Environment API

You control a Unitree G1 humanoid in MuJoCo. Walking and balance are handled by a frozen whole-body controller; you command the base and the upper body. The simulator runs in a separate server process. You talk to it through `harness/client.py`; `run_episodes.py` does this for you.

## Control loop

- Control rate 50 Hz (one `act` call = 0.02 s of simulated time).
- An episode ends on success, on a task-specific failure (an object on the floor, the robot far from the scene), or at the time limit (`obs["time_limit"]` steps). A fall is recorded in `obs["fell"]` but does not end the episode; a fallen robot runs out the clock.
- Success requires the task condition to hold continuously for 1 s (50 consecutive steps).

## Action: 20 floats, physical units

| index | meaning | range |
|---|---|---|
| 0 | `vx` forward velocity command, body frame, m/s | -1..1 |
| 1 | `vy` lateral velocity command, m/s (+ = left) | -1..1 |
| 2 | `height` pelvis height command, m, absolute | 0.4..1.0 (0.74 = normal stance) |
```

```
| 3 | `wz` yaw rate command, rad/s (+ = counter-clockwise) | -1..1 |
| 4..10 | left arm joint targets, rad: shoulder_pitch, shoulder_roll, shoulder_yaw, elbow, wrist_roll, wrist_pitch, wrist_yaw | joint limits |
| 11..17 | right arm joint targets, same order | joint limits |
| 18 | left gripper command | 0..1 (0 = fully open, 1 = fully closed) |
| 19 | right gripper command | 0..1 |
```

Notes on the base controller (important):

- Velocity commands are followed with lag and some wobble. Read the pelvis pose from `low\_dim\_obs` every step and close the loop; do not dead-reckon.
- A planar speed below 0.055 m/s is treated as "stand still". Use 0 or at least 0.06.
- Standing still, the base drifts a few centimetres. Heights above ~0.8 m lock the knees; squatting (lower height) is how you reach low objects. While squatting the controller does not step; walk first, then squat in place.
- Arm targets are absolute joint angles tracked by position control; large jumps are followed over a few tenths of a second. Your commands are applied as given: nothing between you and the robot smooths or rate-limits them.

Observation: dict returned by reset/step

```
| key | shape | meaning |
|---|---|---|
| `t` | int | steps since reset |
| `time_limit` | int | max steps in the episode |
| `fell` | bool | the controller reported a fall at some point in this episode (latched; the episode continues) |
| `low_dim_obs` | (50,) | the proprioceptive vector the learned policies get; layout below |
```

`low\_dim\_obs` layout (50 floats; the learned policies get exactly this vector and nothing else about the body):

```
| index | content |
|---|---|
| 0..6 | left arm joint positions, rad: shoulder_pitch, shoulder_roll, shoulder_yaw, elbow, wrist_roll, wrist_pitch, wrist_yaw (same order as the action) |
| 7..8 | left finger joint positions |
| 9..15 | right arm joint positions, same order |
| 16..17 | right finger joint positions |
| 18..21 | pelvis x, y, z (m, world frame) and yaw (rad, world frame, 0 = +x axis) |
| 22..43 | velocities of entries 0..21, same order (rad/s, m/s) |
| 44..45 | left, right gripper state (0 open .. 1 closed, the scale of the gripper command) |
| 46..49 | pelvis x, y, z, yaw again, as tracked by the base controller |
```

There are no task-specific keys: the observation contains no object positions.

Tools available inside the policy

- `tools.hold\_action()` -> 20 floats: zero base velocity, current height, current arm and gripper targets. Start from this and overwrite what you need.
- `tools.render(camera, path)` saves a PNG from one of the robot's own cameras (`head`, `left\_wrist`, `right\_wrist`; 84x84, the observation itself) under the sandbox and returns its path. Look at the file with your image viewer. Rendering costs no budget.

Running episodes

```
python run_episodes.py # every seed in seeds.json, sequentially
python run_episodes.py --seeds 7,12,14 --jobs 4
```

Per episode it prints success, steps, whether the robot fell and the termination reason, and saves `frames/seed<N>\_last.png` for failed episodes. Records go to `runs/<timestamp>.json`; the policy file is snapshot next to it.

Seeds: the 60 seeds the demonstration set was collected on (also in `seeds.json`): 2, 4, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 30, 32, 33, 34, 35, 36, 38, 39, 40, 41, 42, 43, 44, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69. Any other seed is refused. Each seed fixes the initial placement of the objects. A reset costs 200 steps of budget. The evaluation seed block is reserved and refused by the server.

Information tier: cameras only

The observation does **not** contain object positions. You get your own body state as the same proprioceptive vector the learned policies get (`low\_dim\_obs`, layout in the observation table) and you must perceive the objects from camera images yourself. There is no camera calibration and no kinematic model: like a learned policy, you work from the pixels and the joint readings.

- `tools.image(camera, width=84, height=84)` -> uint8 array (height, width, 3). Cameras: `head`, `left\_wrist`, `right\_wrist`, the three cameras mounted on the robot (the 84x84 views a learned policy gets; this is also the largest size you may request (larger requests are refused), so you and the learned baselines see the same pixels). There are no other cameras: the robot has no external view of itself. Rendering costs no budget but takes a few milliseconds.
- There is no view of the scene other than these three cameras, for you while developing as much as for `policy.py`: to see what the robot sees, save a frame with `tools.render` (or `env.render` in your own scripts) and open it.
- Image processing: numpy, scipy, OpenCV (`cv2`) and pillow are available (the same versions at evaluation); no learned models.

```
## The demonstration
```

One human teleoperation episode of the task, recorded from the robot's own cameras (`demo\_head.mp4`, `demo\_left\_wrist.mp4`, `demo\_right\_wrist.mp4`; 84x84, 25 fps; the control loop ran at 50 Hz, so frame k was recorded at control step 2k). The files are synchronised: frame k of every camera is the same instant. `harness/demo.py` reads them:

```
```python
from harness.demo import Demo
d = Demo()
d.cameras, d.n_frames # ('head', 'left_wrist', 'right_wrist'), frames per camera
im = d.frame("head", k) # uint8 RGB (84, 84, 3), same layout as tools.image()
ims = d.frames("left_wrist") # uint8 RGB (n_frames, 84, 84, 3)
d.step_of(k) # control step at which frame k was recorded
d.frame_at_step("head", 300) # frame closest to control step 300
```
```

This is development material: the demonstration and `harness/demo.py` are not available to `policy.py` at evaluation, and the policy must not read them.

Listing A3: Complete policy.py written by GPT-6 Astra, strict interface, multi-modal reaching (100/100 hidden seeds; verbatim; from the same sessions as Table IV)

```
"""Deterministic color-based visual servo for touching the red sphere.
```

```
The head view selects a hand. The selected wrist view controls shoulder pitch
and yaw, while the base advances slowly. All feedback comes from robot cameras
and proprioception; the constants below are hand-tuned control gains.
"""
```

```
import cv2
import numpy as np
```

```
def red(image):
    """Return red component centroid and enclosing radius, in camera pixels."""
    r = image[:, :, 0].astype(float)
    g = image[:, :, 1].astype(float)
    b = image[:, :, 2].astype(float)
    mask = ((r > 25) & (r > 1.7 * g) & (r > 1.7 * b)).astype(np.uint8)
    count, labels, stats, centers = cv2.connectedComponentsWithStats(mask)
    if count < 2:
        return None
    component = 1 + np.argmax(stats[1:, 4])
    if stats[component, 4] < 3:
        return None

    # Visible area decreases when the camera gets extremely close and clips
    # the sphere. Its outer extent remains a useful stopping signal.
    ys, xs = np.where(labels == component)
    _, radius = cv2.minEnclosingCircle(
        np.column_stack((xs, ys)).astype(np.float32))
    return np.array([*centers[component], radius])
```

```
class Policy:
    def reset(self, obs, tools):
        self.action = np.array(tools.hold_action())
        self.action[18:] = 0.0
        sphere = red(tools.image("head"))
        right = sphere is None or sphere[0] > 41.5
        self.arm = 11 if right else 4
        self.camera = "right_wrist" if right else "left_wrist"
        self.action[self.arm] = (
            np.clip(-0.2 + (sphere[1] - 15) * 0.014, -0.6, 0.1)
            if sphere is not None else -0.35
        )
        self.initial_yaw = obs["low_dim_obs"][21]
        self.initial_y = obs["low_dim_obs"][19]

    def act(self, obs, tools):
        t = obs["t"]
        a = self.action
        if t % 5 == 0:
            sphere = red(tools.image(self.camera))
```

```

body = obs["low_dim_obs"]
a[0] = a[1] = 0.0

# Keep the approach heading and correct lateral walking drift.
dy = self.initial_y - body[19]
if abs(dy) > 0.035:
    a[1] = np.clip(2 * dy, -0.12, 0.12)
a[3] = np.clip(2 * (self.initial_yaw - body[21]), -0.3, 0.3)

if sphere is not None and t > 25:
    x, y, radius = sphere
    a[self.arm] += np.clip((y - 42) * 0.001, -0.025, 0.025)
    a[self.arm] = np.clip(a[self.arm], -0.8, 0.4)
    a[self.arm + 2] += np.clip((42 - x) * 0.0012, -0.04, 0.04)
    a[self.arm + 2] = np.clip(a[self.arm + 2], -0.9, 0.9)

    # Brake early, then allow time for continuous contact. If the
    # initial reach stops short, gradually permit a closer approach.
    stop_radius = min(54, 40 + max(0, t - 350) * 0.06)
    if abs(x - 42) < 22 and abs(y - 42) < 22 and radius < stop_radius:
        a[0] = 0.18 if radius < 25 else 0.08

if abs(a[1]) < 0.055:
    a[1] = 0.0
return a.copy()

```

Listing A4: Excerpt of policy.py written by Claude Opus 5.5, strict interface, multi-modal reaching (100/100 hidden seeds; [...] marks elided lines; full program in the code release). The offsets in fk() are recalled from Unitree's g1_29dof.urdf

```

"""Touch the red sphere with either hand (Unitree G1, scripted).

Pipeline
1. Locate the sphere in the head camera: fit a circle (in ray space) to the
red blob boundary -> direction + angular radius -> 3D point, using a
pinhole model of the head camera calibrated from the floor checkerboard.
2. Store the target in world coordinates (pelvis odometry) and walk the base
so the target sits at a comfortable spot in front of one shoulder.
3. Re-observe, then reach with an analytic G1 arm model + numerical IK:
sweep the hand forward through the estimate; the sphere lights up while
touched, so the lit stretch of the sweep gives the touch-zone centre
(then z / y sweeps if needed), and the hand holds there.
The arms hang down (out of the head camera view) until the reach starts.
"""

# [...]
# ----- camera model
F = 72.746 # focal length in px (fovy 60 deg, 84 px)
CX = CY = 41.5
CAM_PITCH = 1.093 # rad below horizontal
CAM_OFF = np.array([0.07, 0.0, 0.40]) # head camera position in pelvis frame
R_SPHERE = 0.047 # effective sphere radius for the boundary detector
# [...]

def fit_sphere(im):
    # [...]
    rays = np.stack([(pts[:, 0] - CX) / F, (pts[:, 1] - CY) / F, np.ones(len(pts))], 1)
    rays /= np.linalg.norm(rays, axis=1, keepdims=True)

    def dirf(p):
        d = np.array([p[0], p[1], 1.0])
        return d / np.linalg.norm(d)
    # [...]
    def res(p):
        return np.arccos(np.clip(rays @ dirf(p[2:]), -1, 1)) - p[2]

    r = least_squares(res, x0)
    return dirf(r.x[2:]), r.x[2] + 0.5 / F, len(pts)

def locate(im):
    """Sphere centre in the pelvis frame (x fwd, y left, z up rel. pelvis)."""
    s = fit_sphere(im)
    if s is None:
        return None
    d, alpha, npts = s
    if alpha <= 0.01:
        return None
    D = R_SPHERE / np.sin(alpha)

```

```

    return R_CAM @ (D * d) + CAM_OFF, npts

# ----- arm model
def rx(a):
    c, s = np.cos(a), np.sin(a)
    return np.array([[1, 0, 0], [0, c, -s], [0, s, c]])

def ry(a):
    c, s = np.cos(a), np.sin(a)
    return np.array([[c, 0, s], [0, 1, 0], [-s, 0, c]])

def rz(a):
    c, s = np.cos(a), np.sin(a)
    return np.array([[c, -s, 0], [s, c, 0], [0, 0, 1]])

TORSO = np.array([-0.0039635, 0, 0.054])
Q16 = 2 * np.arcsin(0.139201)
TOOL = np.array([0.10, 0.0, 0.0])

def fk(q, side):
    """Tool point and wrist rotation (pelvis frame) for arm joints q (7)."""
    s = side
    R = np.eye(3)
    p = TORSO + np.array([0.0039563, s * 0.10022, 0.23778])
    R = rx(s * Q16) @ ry(q[0])
    p = p + R @ np.array([0, s * 0.038, -0.013831])
    R = R @ rx(-s * Q16) @ rx(q[1])
    p = p + R @ np.array([0, s * 0.00624, -0.1032])
    R = R @ rz(q[2])
    p = p + R @ np.array([0.015783, 0, -0.080518])
    R = R @ ry(q[3])
    p = p + R @ np.array([0.1, s * 0.00188791, -0.01])
    R = R @ rx(q[4])
    p = p + R @ np.array([0.038, 0, 0])
    R = R @ ry(q[5])
    p = p + R @ np.array([0.046, 0, 0])
    R = R @ rz(q[6])
    return p + R @ TOOL, R

IDX = [0, 1, 2, 3, 5] # joints used by the IK: shoulder pitch/roll/yaw, elbow, wrist pitch

# sane working ranges (left arm) for sp, sr, sy, el, wp; roll/yaw mirrored for the right arm
W_LO = np.array([-1.9, 0.0, -0.9, -0.9, -1.0])
W_HI = np.array([0.5, 0.9, 0.9, 1.6, 1.0])

def limits(side):
    lo, hi = W_LO.copy(), W_HI.copy()
    if side < 0:
        for i in (1, 2):
            lo[i], hi[i] = -W_HI[i], -W_LO[i]
    return lo, hi

def ik(p, side, q0=None, approach=np.array([1.0, 0, 0]), w_ori=0.05):
    lo, hi = limits(side)
    if q0 is None:
        starts = [np.clip(np.array(s, float) * np.array([1, side, side, 1, 1]), lo + 1e-3, hi - 1e-3)
                  for s in [(-0.3, 0.1, 0, -0.5, 0), (-0.8, 0.1, 0, -0.3, 0), (-0.5, 0.2, 0, 0.3, 0),
                           (-1.2, 0.1, 0, 0.3, 0), (0, 0, 0, 0, 0)]]
    else:
        starts = [np.clip(np.asarray(q0)[IDX], lo + 1e-3, hi - 1e-3)]

    def res(x):
        q = np.zeros(7)
        q[IDX] = x
        tip, R = fk(q, side)
        return np.r_[(tip - p) * 30, w_ori * (R[:, 0] - approach), 0.003 * x]

    best = None
    for s0 in starts:
        r = least_squares(res, s0, bounds=(lo, hi))
        if best is None or r.cost < best.cost:
            best = r
    q = np.zeros(7)
    q[IDX] = best.x
    return q, float(np.linalg.norm(fk(q, side)[0] - p))

```